

一、核心语法

1. 数据存储与运算

1.1 字面量

字面量：程序中直接书写的固定值(数据)。

字面量类型及书写格式

字面量类型及书写格式			
数字类型	整数(int)	10 18 -5 0	
	浮点数/小数(float)	8.5 3.14 1.0 -3.5	
布尔(bool)	表达现实生活中的逻辑，真或假	True False	
字符串(str)	描述文本的一种数据类型	"人生苦短，我用Python"	
空值(NoneType)	表示空或无值，仅包含一个值None	None	
数据容器	存储多项数据的容器	列表/元组/集合/字典 (后面详细讲解)	

说明：布尔类型本质就是数字类型，在涉及到数学运算时，会自动将True转为1，False转为0。
小结：

1.什么是字面量？

•程序中，直接书写的固定值（数据）

2.各类型字面量的书写形式？

•整数(int)/小数(float)：直接写，如 10 3.14

•字符串(str)：双引号("人生苦短, 我用Python")

•布尔(bool)：True、False

•空值(NoneType)：None

1.2 变量

变量：程序中用来存储单个数据的容器，通常会把经常发生变化的数据存储在变量中。

注意：变量是指存储数据的容器（空间），而不是容器里面存储的数据。

小结：

1.什么是变量? 如何定义？

•程序中用来存储单个数据的容器（经常会发生改变的数据）； num = 10

2.变量的使用？

•输出打印 / 参与计算 / 记录数据

3.注意事项？

- 一个变量只能存储一个值
- 变量定义的时候必须赋值才可以使用
- 一条语句可以定义多个变量，也可以连续赋值（a,b = 1,"Python"）

1.3 标识符

标识符：程序员在代码中为变量、函数、类等元素所起的**名字**。

小结：

1.标识符的命名规则(规定)?

- 只能包含字母（a-z, A-Z）、数字（0-9）、下划线（_）
- 数字不能开头
- 不能使用关键字：if、else、elif、for、while、and、or等
- 严格区分大小写，比如：age, Age, AGE是三个变量

2.变量命名规范

- 见名知意
- 多个部分使用下划线连接
- 英文字母全小写

1.4 数据类型

常见数据类型

标准名称	描述	说明
int	整数	数字类型，存放整数，如：10 -5
float	浮点数	数字类型，存放小数，如：8.5 3.14 1.0 -3.5
str	字符串	用引号引起来的都是字符串，如："Python"
bool	布尔	布尔类型，描述真和假，如：True False
NoneType	空值	表示空或无值，仅包含一个值None

查看数据类型的方法：

- 1.通过type()语句来得到数据的类型，具体语法为：type(要查看类型的数据)
- 2.通过isinstance()检查数据是否属于指定的类型，返回的是一个bool值，具体语法为：isinstance(数据，类型)

说明：变量本身是没有类型的，type(变量)输出的类型是变量中存储的数据的类型。

小结：

1.常见的数据类型有哪些？

int、float、str、bool、NoneType

2.如何查看数据的实际类型？

type(...)

isinstance(...)

1.5 字符串

字符串的三种定义方式

```
s1 = "hello" # 双引号
s2 = '你好' # 单引号
s3 = """
hello:
    welcome!
""" # 三引号
```

常见的转义字符

转义字符	名称	作用
\'	单引号	表示单引号 '
\"	双引号	表示双引号 "
\n	换行符	开始新的一行(换行)
\t	制表符	增加缩进，缩进一个制表符（tab）的大小

小结:

1.字符串的定义方式?

- 单引号 ('Python') 、双引号("Python")、三引号 ("""Python""")

2.常见的转义字符?

\' (单引号)、\" (双引号)、\n (换行符)、\t (制表符)

3.字符串的使用原则?

- 单引号 与 双引号是等效的，项目中保持一种写法即可。
- 三引号：多行字符串等场景

1.6 字符串拼接

两种拼接方法

```
# 字符串拼接
s1 = "人生苦短" ", 我用Python" #将多个字符串字面量直接写
print(s1)

msg1 = "人生苦短"
msg2 = "我用Python"
print("龟叔说: " + msg1 + " , " + msg2) # +号拼接
```

注意：+ 号可以用来拼接两个字符串，但是无法将非字符串与字符串进行拼接（非字符串类型需要转换为字符串类型）。

str用法：

str(int)----->将int类型的数字转为字符串类型

1.7 字符串格式化

1.通过 %占位符 的形式完成字符串和变量的快速拼接。（其中 % 表示我要占位，s表示将变量转为字符串放入占位的位置）

2.可以通过 f'内容{变量/表达式}' 的形式来完成快速格式化。（推荐）

注意：前面有多少个占位符（%s），后面就需要有多少个变量(或数据)，前后数量需要一致。

案例：

```
# 案例
name = "小鱼"
age = 22
sub = "计算机技术"
hobby = "Python, Java"
print("大家好，我是" + name + "，今年" + str(age) + "岁，" + "学习的专业是" + sub + "，爱好是" + hobby)

# 字符串格式化
print("大家好，我是%s，今年%s岁，学习的专业是%s，爱好是%s" % (name, str(age), sub, hobby))
print(f"大家好，我是{name}，今年{age}岁，学习的专业是{sub}，爱好是{hobby}")
```

小结

1.字符串拼接使用什么符号? 字符串能否和非字符串拼接?

- +
- 不能，只能拼接字符串，其他类型需要转换为字符串（str(..））

2.% 这种字符串格式化的语法?

- 语法："%s " % 变量 / "%s..%s" % (变量,变量)

3.f 这种字符串格式化的语法?

- 语法：f'内容{变量/表达式}'
- 推荐

1.8 输入与输出

input语句（函数）的功能就是获取键盘输入的数据，具体的用法为：s = input(提示信息)

print语句(函数)的功能就是将数据输出到控制台，具体语法为：print(数据..)

注意：无论键盘输入什么类型的数据，获取到的数据永远都是字符串类型。

小结：

1.如何获取用户输入的数据?

- s = input(提示信息)

2.如何进行数据类型转换，比如字符串转为数字、数字转字符串等？

- 其他类型转为int类型：int(..)
- 其他类型转为str类型：str(..)
- 其他类型转为float类型：float(..)
- 其他类型转为bool类型：bool(..)

1.9 算术运算符

类型

运算符	描述	举例说明
+	加法	1 + 5 : 1加上5
-	减法	8 - 3 : 8减3
*	乘法	3 * 8 : 3乘以8
/	除法	10 / 5 : 10除以5，除法结果是小数
//	整除	10 // 3 : 10整除3，整除结果为整数
%	取余/求模	10 % 3 : 10模于3，结果为1（10除3取余数）
**	幂指数	10 ** 3 : 10的3次方

注意：由于计算机底层是基于二进制来进行数据的存储和处理的，而二进制是无法精确表示所有的小数的，因此涉及到浮点数的运算时，可能损失精度。

1.10 赋值运算符

类型

运算符	描述	实例
=	赋值运算符	把 = 号右边的结果赋值给左边的变量，如 num = 1+2，结果num的值为3
+=	加法赋值运算符	num += 2 等效于 num = num + 2
-=	减法赋值运算符	num -= 2 等效于 num = num - 2
*=	乘法赋值运算符	num *= 2 等效于 num = num * 2
/=	除法赋值运算符	num /= 2 等效于 num = num / 2
%=	取模赋值运算符	num %= 2 等效于 num = num % 2
//=	取整除赋值运算符	num //= 2 等效于 num = num // 2
**=	幂赋值运算符	num **= 2 等效于 num = num ** 2

1.11 比较运算符

比较运算符：也称为关系运算符，用于比较两个值之间的关系。会计算运算符两边的表达式，然后返回一个布尔值作为结果(True – 表示比较关系成立； False – 表示比较关系不成立)。

类型：

运算符	描述	实例
==	等于	a == b 判断a是否等于b
!=	不等于	a != b 判断a是否不等于b
>	大于	a > b 判断a是否大于b
>=	大于等于	a >= b 判断a是否大于等于b
<	小于	a < b 判断a是否小于b
<=	小于等于	a <= b 判断a是否小于b

1.12 逻辑运算符

逻辑运算符：用于连接多个条件(布尔)表达式（其值为“真”或“假”），并返回一个最终布尔结果的运算符。

类型：

运算符	描述	实例
and	逻辑与(并且)	同时成立才是符合条件的(左右两边都为True，结果才为True)
or	逻辑或(或者)	只要有一个符合要求的即可(只要左右两边有一个为True，结果就为True)
not	逻辑非(取反)	取反操作，True变为False， False变为True

2. 流程控制语句

2.1 if 语句

格式：

if条件判断

- 场景：只有满足指定条件，才会执行对应的代码逻辑。

if条件判断基本格式

if 要判断的条件:

条件成立时，要执行对应的操作

判断语句的结果，必须是布尔类型True或False
True 会执行if内的代码
False则不会执行

小结：

1.if条件判断的基本格式

if 要判断的条件:

条件成立时, 要执行对应的操作1

条件成立时, 要执行对应的操作2

2.if语句的注意事项

- 判断条件的结果一定要是布尔类型
- 不要忘记判断条件后的冒号(:)
- if语句里面的代码块, 需要在前方缩进空格(建议4个空格), 通过缩进来描述代码的层级关系(归属)

语句进阶:

if - else 语句

小结:

1.if...else 语句的语法

if 要判断的条件:

条件成立时, 执行对应的操作1

else:

条件不成立时, 执行的操作2

2.if...else 语句注意事项

- else是不需要条件判断的, 当if条件不成立时, else就执行了
- else代码块, 也需要使用空格缩进(建议4个空格)

if - elif -else语句

```
# if...elif...else结构

if 要判断的条件1:
    条件成立时, 执行对应的操作1 ①

elif 要判断的条件2:
    条件成立时, 执行对应的操作2 ②

else:
    条件不成立时, 执行的操作3 ③
```

```
# if...elif...else结构

num = int(input("请输入数字: "))
if num > 0:
    print(f"{num} 是 正数")
elif num < 0:
    print(f"{num} 是 负数")
else:
    print(f"{num} 是 0")
```

注意：满足第1个条件，就不会判断第2个部分和第3个部分；第1个条件不成立，才会判断第2个条件；1和2都不成立，才会进入else。

小结：

1.if...elif...else语句的作用

- 可以完成多个条件的判断(多分支判断)

2.if...elif...else语句注意事项

- elif 可以写多个
- else 可有可无，如果有必须放在最后
- 多个条件判断是有顺序的，从上到下判断，如果前面有条件成立了，后面的条件不会判断了

2.2 match模式匹配

执行流程

模式匹配 match...case

```
# 工作日程安排
day = input("请输入星期几(1-7): ")
match day:
    case "1":
        print("周一：工作会议日")
    case "2":
        print("周二：学习培训日")
    case "3":
        print("周三：项目开发日")
    case "4":
        print("周四：代码审查日")
    case "5":
        print("周五：总结规划日")
    case "6" | "7":
        print("周末：休息放松")
    case _:
        print("输入错误")
```

执行流程：

1. 首先计算match指定的表达式的值。
2. 从上到下依次和case后面的值进行匹配，匹配正确，就执行对应语句。
3. 如果前面所有的case都没有匹配上，就会执行默认的case _。

其中的 | 表示或的关系，匹配多个模式中的任意一个

匹配其他所有情况

小结：

1.match...case语法

match 表达式：

case 值1：

操作1

case 值2 if 条件表达式：

操作2

case 值3 | 值4：

操作4

case _：

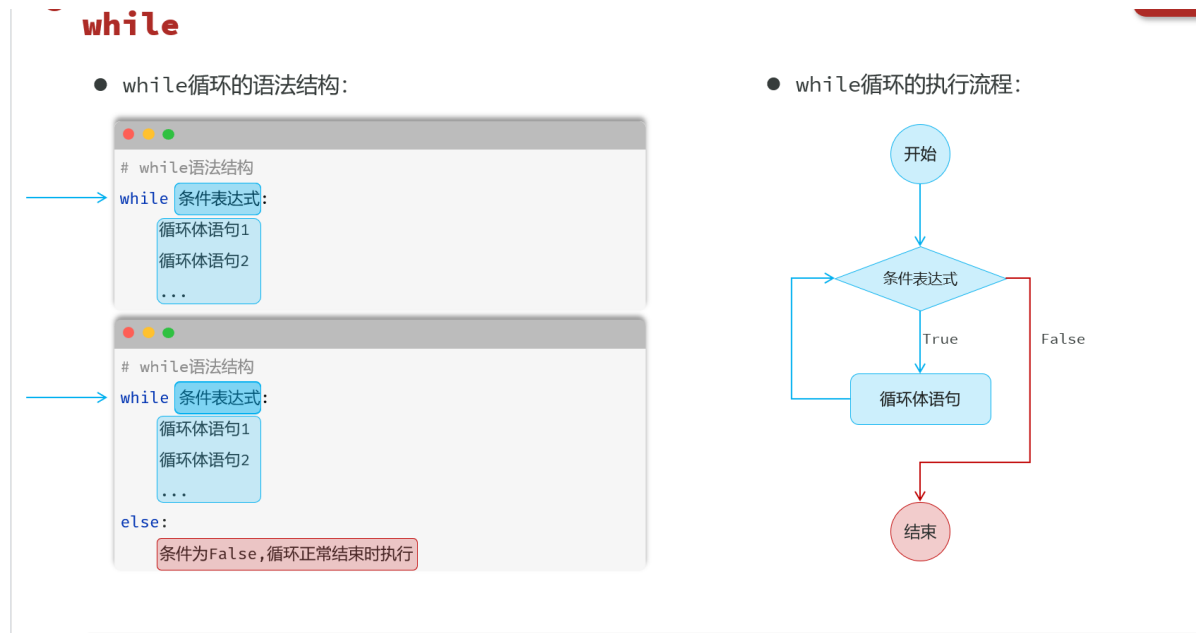
默认操作

2.match...case应用场景

- match：基于某个变量的多个固定值进行分支判断时，可以使用match模式匹配
- if：条件判断涉及复杂的逻辑判定、范围比较及组合条件时

2.3 while语句

语法结构：



注意：while循环是通过条件表达式，来控制是否要进行下一次循环。

小结：

1.while循环的语法格式

while 条件表达式：

条件成立时，执行的操作（循环体语句）

条件成立时，执行的操作（循环体语句）

.....

else:

循环正常结束时，执行的操作

可选

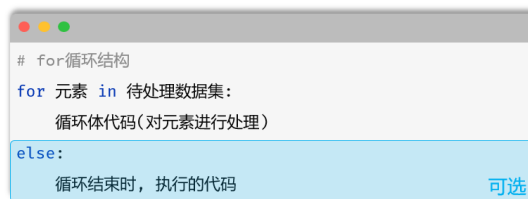
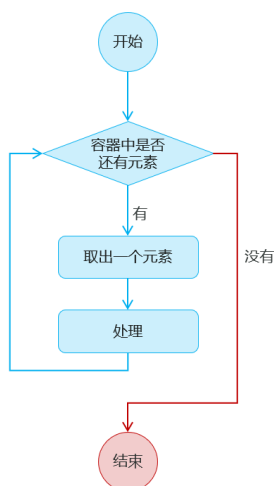
2.while循环的注意事项

- 条件表达式的结果为布尔类型
- 通过空格缩进表述层级关系
- 规划好循环终止的条件，否则将无限循环（死循环）

2.4 for循环

for循环

- while循环是通过条件表达式来控制是否要进行下一次循环的。而for循环，本质是一种**轮询遍历**机制，对一批内容进行逐个处理。



遍历字符串 "Hello-Python"



小结:

1.for循环的语法格式

for 元素 in 待处理数据集:
循环体代码(对元素进行处理)

else:

不满足循环条件时, 执行的操作

可选

2.for循环与while循环的场景

- while循环: 用于在某个条件满足时一直循环, 循环的次数通常是未知的, 只知道循环开始/结束的条件。(关注的是循环的条件)
- for循环: 用于对一个已知的数据集进行遍历或已知次数的循环。(关注的是遍历每一个元素)

2.5 range语句

- 作用:** 生成指定规则的数字序列
- 用法1:** `range(end)` -> 获取一个从0开始, 到end结束的数字序列 (不含end本身)
 - `range(5)` 获取的数据就是 0,1,2,3,4
- 用法2:** `range(start,end)` -> 获取一个从start开始, 到end结束的数字序列 (不含end本身)
 - `range(2,8)` 获取的数据就是 2,3,4,5,6,7
- 用法3:** `range(start,end,step)` -> 获取一个从start开始, 到end结束的数字序列, step步长 (不含end本身)
 - `range(0,10,2)` 获取的数据就是 0,2,4,6,8

print的特点

`print("*")` : 自带换行效果 , 每一次执行都会输出新的一行中 ;
`print("*", end="")`: `end`表示的是每一次输出以什么结束; 默认 `\n`, 表示换行 .

2.6 break和continue

关键字:

`break`: 只能够出现在循环中, 表示结束、跳出循环的含义 (`break`跳出循环时, `while`后面的`else`中的代码将不会执行)

`continue`: 只能够出现在循环中, 表示中断本次循环, 直接进入下一次循环

break关键字的作用?

• 不能够单独书写, 只能出现循环中, 表示结束、跳出的意思。

continue关键字的作用?

• 不能够单独书写, 只能出现循环中, 表示中断本次循环, 直接进入下一次循环。

2.7 生成随机数字

生成1到100的随机数 (包含1和100)

```
import random
random_num = random.randint(1,100)
```

3. 数据容器

3.1 列表list

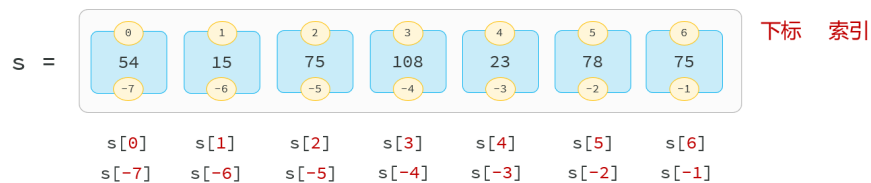
● 列表是数据容器中的一类, 是一性可以存储多个数据 (元素) 的。

● 定义: 列表名称 = [元素1, 元素2, 元素3, 元素4, 元素5...]

`s = [54, 15, 75, 108, 23, 78, 75]`

● 特点:

- 可以存储不同类型的元素
- 元素有序、可以重复、元素可以修改



序列类型: 容器中的元素按特定顺序排列的、可通过索引访问的容器类型称之为序列。

注意: 从前向后 (正向索引), 下标从0开始; 从后向前 (反向索引), 下标从-1开始。

小结:

1.列表的定义方式及特点

• 列表名 = [元素1, 元素2, 元素3]

• 可以存放不同类型元素、可重复、有序、元素可以修改

2.列表的索引

• 正向索引: 从0开始

•反向索引：从-1开始

3.列表元素的查看、修改、删除

•查看：list1[0]

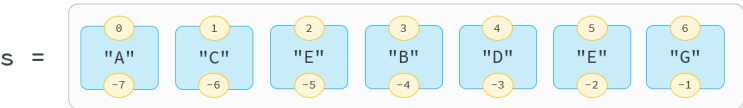
•修改：list1[0] = "A"

•删除：del list1[3]

•注意：如果指定的索引值超出范围，将会报错

切片

- 介绍：切片是指对操作的数据截取其中一部分的操作。列表、字符串、元组都支持切片操作。



- 语法：序列数据[开始索引:结束索引:步长]
 - 不包含结束索引位置对应的元素(开始索引未指定默认为0；结束索引未指定默认为列表长度(直到列表末尾)；步长未指定默认为1)
 - 索引采用正向、反向索引都可以
 - 步长是选取间隔，默认步长为1
- 举例：
 - s[0:5:1] 切片后的结果为 ["A", "C", "E", "B", "D"]
 - S[0:5:2] 切片后的结果为 ["A", "E", "D"]

小结：

1.列表的切片

•语法：

```
s[start:end:step]
```

- 特点：
- start：开始索引，不指定默认为0（第一个元素的索引）
- end：结束索引，不指定默认为列表长度(直到列表末尾)
- step：步长，不指定默认为1

列表的常见方法

方法	作用	样例
append()	在列表的尾部追加元素	s.append(10086)
insert()	在指定索引之前，插入该元素	s.insert(0, 92)
remove()	移除列表中第一个匹配到的值	s.remove(75)
pop()	删除列表中指定索引位置的元素（如果未指定索引，默认删最后一个）	s.pop(2) / s.pop()
sort()	对列表进行排序（列表元素的数据类型一致，才可以进行排	s.sort()

sort() 方法	作用	s.sort() 样例
reverse()	反转列表元素	s.reverse()

sum()----->进行求和
len()----->获取元素的个数
求平均值 sum(s)/len(s)

小结:

Python中用于数据统计的常见语句

- min(): 获取最小值
- max(): 获取最大值
- sum(): 求和
- len(): 获取元素的个数

小结:

1.如何快速合并两个列表?

- 使用+运算符直接合并
- 使用*进行解包操作(解包指将列表这一类数据容器解开成独立的元素)

```
num_list1 = [19, 23, 54, 64, 875, 20, 109, 232, 123, 54]
num_list2 = [55, 80, 72, 35, 60, 123, 54, 29, 91]

num_list = num_list1 + num_list2
num_list = [*num_list1, *num_list2]
```

2.如何判断一个元素是否存在于列表之中?

- 可以使用in运算符, 语法为: 元素 in 列表
- 返回结果为布尔值, True表示存在, False表示不存在

```
new_list = []
for num in num_list:
    if num not in new_list:
        new_list.append(num)
```

小结:

1.列表推导式

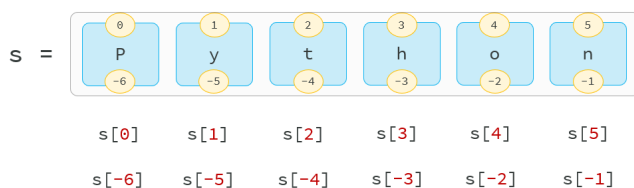
- 含义: 列表推导式就是按照一定规则快速生成一个列表的方法
- 格式1: 列表名称=[要插入列表的数据 for i in 列表]

•格式2：列表名称=[要插入列表的数据 for i in 列表 if 条件]

```
squ_list = [i**2 for i in range(1,21)]  
new_list1 = [i ** 2 for i in num_list if i % 2 == 0]
```

3.2 字符串str

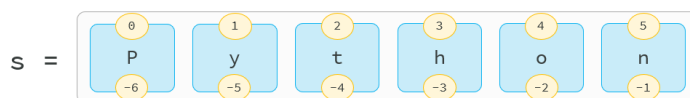
- 介绍：字符串是字符的容器，一个字符串中可以存放任意数量的字符。如："Python" 'Python' """Python"""
- 特点：不可变性（无法修改）、有序性、可迭代性。
- 字符串中的每一个字符元素都有其对应的下标（索引），通过元素对应的索引，就可以获取到对应的元素。



注意：从前向后（正向索引），下标从0开始；从后向前（反向索引），下标从-1开始。

字符串-切片

- 介绍：切片是指对操作的对象截取其中一部分的操作。字符串、列表、元组都支持切片操作。



- 语法：序列对象[开始索引:结束索引:步长]
 - 不包含结束索引位置对应的元素(开始索引未指定默认为0；结束索引未指定默认为-1；步长未指定默认为1)
 - 索引采用正向、反向索引都可以
 - 步长是选取间隔，默认步长为1（如果是-1，表示从后向前）
- 举例：
 - s[0:5:1] 切片后的结果为 "Pytho"
 - s[0:5:2] 切片后的结果为 "Pto"

小结：

1.字符串的特点

•不可变性（无法修改）、有序性、可迭代性

2.字符串的索引

•正向索引：从0开始

•反向索引：从-1开始

3.字符串的切片

•语法：

```
s[start:end:step]
```

•特点：

•start：开始索引，不指定默认为0（第一个元素的索引）

•end：结束索引，不指定默认为-1（最后一个元素的索引）

•step：步长，不指定默认为1（-1表示从后向前）

字符串操作常用方法

方法	作用	样例
find()	在字符串中查找子串，返回第一次出现的索引位置，找不到返回-1	s.find('Python')
count()	统计子串在字符串中出现的次数	s.count('H')
upper()	将字符串中的所有字母转换为大写	s.upper()
lower()	将字符串中的所有字母转换为小写	s.lower()
split()	将字符串按指定分隔符分割成列表	s.split(' ')
strip()	去除字符串两端的空白字符或指定字符	s.strip() / s.strip('*')
replace()	将字符串中的指定子串替换为新的子串	s.replace('H','C')
startswith()	检查字符串是否以指定子串开头，返回布尔值	s.startswith('P')

3.3 元组tuple

● 介绍：元组是**不可变**的序列，类似于列表，但创建后不能修改。

● 特点：

- 可以存储不同类型的元素
- 元素可以重复、有序、**不可以修改**（支持索引访问、切片）

● 定义：

```
# 定义元组
元组名称 = (元素1, 元素2, ...)
```

```
# 定义空元组
元组名称 = ()
元组名称 = tuple()
```

```
# 定义元组
t1 = (5, 7, 9, 1, 2, 3, 10, 6, 4, 8, 12, 7, 5)
```

```
# 定义空元组
t2 = ()
t3 = tuple()
```

● 方法：

- count(): 统计某元素在元组中出现的次数
- index(): 查找某个元素在元组中的索引位置（第一次出现的位置）

小结：

1.元组有什么特点？

•元素可以重复

•有序

•不可修改（只读）

2.元组的两个查询方法？

•count(): 统计指定元素的数量

•index(): 获取指定元素的索引，如果该元素不存在将报错

3.元组使用注意事项？

- 定义单元素元组时，需要在结尾加上逗号，比如 ('A,')

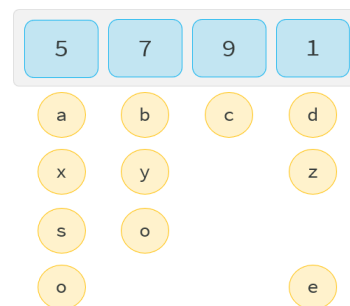
组包与解包

- 组包 (Packing)：将多个值合并到一个容器（元组、列表）中。
- 解包 (Unpacking)：将容器（元组、列表）解开成独立的元素，分别赋值给多个变量。

```
# 定义元组，组包
t1 = (5, 7, 9, 1)
# 定义元组，组包
t2 = 5, 7, 9, 1

# 基础解包
a, b, c, d = t1

# (*)扩展解包
x, *y, z = t2 # x为5, y为[7, 9], z为1
s, *o = t2 # s为5, o为[7, 9, 1]
*o, e = t2 # o为[5, 7, 9], e为1
```



说明：在元组解包时，* 表示收集剩余的所有元素，允许我们处理不确定数量的元素（生成列表，以便于可以进行进一步的处理）。

小结：

组包与解包？

- 组包 (Packing)：将多个值合并到一个容器（元组、列表）中。
- 解包 (Unpacking)：将容器中的元素分别赋值给多个变量。
- 在元组解包时，* 表示收集剩余的所有元素，允许我们处理不确定数量的元素

控制浮点数生成位数

```
print(f"平均分为: {avg:.1f}")
```

3.4 集合set

- 介绍：集合(set)是一种无序的、不可重复、可修改的数据容器。
- 定义：

```
set集合定义

# 定义集合
s1 = {"C", "D", "X", "T", "O", "U"}

# 定义空集合
s2 = set()
```

注意：空集合的定义不可以使用 {}, {} 表示的是空字典；由于集合是无序的，因此是不支持下标索引访问的。

集合常用方法

操作	含义	样例
<code>add(..)</code>	添加元素到集合中	<code>s1.add('t')</code>
<code>remove(..)</code>	移除集合中的指定元素 (指定元素不存在将报错)	<code>s1.remove('t')</code>
<code>pop()</code>	随机删除集合中的元素并返回	<code>e = s1.pop()</code>
<code>clear()</code>	清空集合	<code>s1.clear()</code>
<code>difference()</code>	求取两个集合的差集 (包含在第一个集合但不包含在第二个集合的元素)	<code>s1.difference(s2)</code>
<code>union()</code>	求取两个集合的并集	<code>s1.union(s2)</code>
<code>intersection()</code>	求取两个集合的交集	<code>s1.intersection(s2)</code>

小结:

1.集合(set)的特点?

•无序、不可重复、可修改 (不要依赖显示顺序)

2.集合(set)的定义及常用操作

•定义1: 集合名 = {元素1,元素2,元素3...}

•定义2: 集合名 = set()

•常用操作:

•添加: `s1.add(..)`

•删除: `s1.remove(..)` / `s1.pop()` / `s1.clear()`

•交、并、差集: `s1.intersection(s2)` / `s1.union(s2)` / `s1.difference(s2)`

注意: 求并集的时候可以用 `|` , 求交集的适合可以用 `&` , 求差集的时候可以用 `-`
集合推导式:

```
set = {需要的数据 for s in all_set if 条件}
```

小结:

1.集合set操作的运算符 & | -

&: 交集

|: 并集

-: 差集

2.集合推导式的写法

变量名称={i表达式 for i in 列表}

变量名称={i表达式 for i in 列表 if 条件}

3.5字典dict

- 字典：使用键值对（key: value）来存储数据，每一个键都对应一个值，通过键（key）可以快速找到对应的值（value）。
- 特点：键值对（key: value）存储、键（key）不能重复、可修改。
- 定义：

```
# 定义字典
字典名称 = {key: value, key:value, key:value ...}

# 定义空字典
字典名称 = {}

字典名称 = dict()

# 根据key获取value
值 = 字典名称[key]
```

```
# 定义字典
dict1 = {"王林": 675, "李慕婉": 608, "许立国": 478, ...}

# 定义空字典
dict2 = {}

dict3 = dict()

# 根据key获取value
score = dict1["李思"]
```

注意：字典（dict）中的value可以是任何类型的数据，而key不能为可变类型（如：不能为 列表list、集合set、字典dict）。

小结：

1.字典的存储形式及特点？

- 字典(dict)存储的是键值对(key:value)形式的数据
- 特点：键值对、key是唯一的、可修改

2.字典定义的语法？

- 定义：dict1 = {"王林": 675, "李慕婉": 608, "许立国": 478, ...}
- 获取：v = dict1["王林"]

3.字典的注意事项

- value可以是任意类型，而key必须是不可变类型（不能为list、set、dict）
- 字典内的key不允许重复，如果重复定义，后面的覆盖前面的
- 字典是没有索引下标的，不能根据索引获取值，只可以根据key获取value

常用操作：

类型	操作	含义	样例
添加	字典名称[key] = value	往指定字典中添加key-value键值对	dict1["涛哥"] = 688
删除	字典名称.pop(key)	删除字典中指定的key，并返回该key对应的value	score = dict1.pop("涛哥")
	del 字典名称[key]	删除字典中指定的键值对	del dict1["涛哥"]
修改	字典名称[key] = value	修改字典中指定的key对应的值	dict1["小智"] = 658
查询	字典名称[key]	根据key获取value	dict1["涛哥"]
	字典名称.get(key)	根据key获取value	dict1.get("涛哥")
	字典名称.keys()	获取所有的key	dict1.keys()
	字典名称.values()	获取所有的value	dict1.values()
	字典名称.items()	获取所有的key-value键值对	dict1.items()

小结：

1.字典的常用操作？

- 添加：字典[key] = value (key不存在，就会执行新增)

- 删除：del 字典[key] / value = 字典.pop(key)
- 修改：字典[key] = value (key存在，就会执行修改)
- 查询：字典[key] / 字典.get(key) ; 字典.keys() / 字典.values() / 字典.items()

2.字典的遍历？

- 字典支持for循环遍历
- 通过keys()、values()、items() 进行循环遍历

数据容器总结与对比

特性	字符串 (str)	列表 (list)	元组 (tuple)	集合 (set)	字典 (dict)
有序性	有序	有序	有序	无序	有序(3.7+)
重复元素	允许	允许	允许	不允许	key不允许
可变性	不可变	可变	不可变	可变	可变
索引访问	支持	支持	支持	不支持	不支持
切片操作	支持	支持	支持	不支持	不支持
使用场景	文本处理	有序可重复数据集	固定数据记录	去重数据集	键值对

4. 函数

4.1 函数基础

函数：是组织好的、可重复使用的、用来实现特定功能的代码片段。

- 函数是组织好的、可重复使用的、用来实现特定功能的代码片段。
- 函数定义与调用：

```
# 定义函数
def 函数名(参数列表):
    函数体
    .....
    return 返回值

# 调用函数
函数名(参数)
```

```
# 定义函数
def out_line():
    print('-----')

# 调用函数
out_line()
```

注意：函数定义时的参数列表与返回值语句是可有可无的(由需求确定)，函数必须先定义后调用执行（函数定义时，内部代码并未执行，在调用时才执行）。

小结：

1.函数定义及调用的语法？

```
# 函数的定义

def out_line():

    print('-----')

# 函数的调用

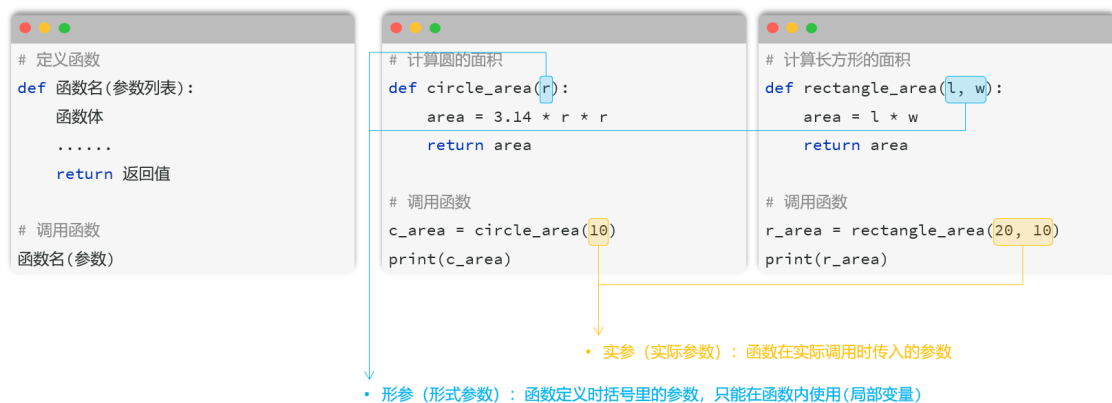
out_line()
```

2.函数使用的注意事项

- 函数必须先定义，在调用
- 函数定义时，并不会执行，只有在调用函数时，函数体的逻辑才会运行
- 函数中通过缩进来描述归属关系

函数的参数与返回值

- 在定义函数时，根据业务需要，可以指定参数与返回值，具体格式如下：



注意：函数定义时如果有多个参数，多个参数之间使用逗号 (,) 分隔。

注意：return语句只有返回功能，而没有输出打印的功能，如果要输出，需要结合print()函数来实现。

小结：

1. 有参数有返回值的函数定义及调用？

```
def rectangle_area(l, w):  
    return l * w  
  
r_area = rectangle_area(l: 20, w: 10)  
print(r_area)
```

形参 → 实参

2. 函数可以有多个返回值吗？

```
def circle_area_len(r):  
    return 3.14 * r * r, 2 * 3.14 * r  
  
al = circle_area_len(10)  
print(al)
```

封装到元组中

```
def circle_area_len(r):  
    return 3.14 * r * r, 2 * 3.14 * r  
  
area, len = circle_area_len(10)  
print(area, len)
```

元组解包

函数的说明文档

- 函数的说明文档 (Docstring) 是写在函数开头，用三个引号包裹的字符串，用于解释函数的功能、参数、返回值等信息，方便调用者清楚函数的具体作用及细节。

```
# 定义一个函数，根据半径，计算圆的周长、面积  
def circle_area_len(r):  
    """  
    该函数用于根据圆的半径，计算圆的面积和圆的周长  
    :param r: 圆的半径  
    :return: 圆的面积，圆的周长  
    """  
    return 3.14 * r * r, 2 * 3.14 * r  
  
al = circle_area_len(10)  
print(al)
```

→ 描述函数的返回值
→ 描述函数的参数

查看函数说明文档：

- 使用help函数，比如：help(circle_area_len)
- 鼠标悬浮在函数上，自动展示（推荐）

```
al = circle_area_len(10)  
print(al)
```

PEP 8: E305 expected 2 blank lines after class or function definition, found 1
Reformat the file Alt+Shift+Enter More actions... Alt+Enter

D:\py_code_workspace\py_project1\04_函数.py
def circle_area_len(r: Any) -> tuple[float | Any, float | Any]

该函数用于根据圆的半径，计算圆的面积和圆的周长

Params: r - 圆的半径

Returns: 圆的面积，圆的周长

记住：好的文档，能让你的代码更容易理解、使用和维护！

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO (Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")  
  
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")  
  
def function_c():  
    print("c ...")  
  
function_a()
```



function_a() 开始

- 打印 "a ... before"
- 调用 function_b()
 - 打印 "b ... before"
 - 调用 function_c()
 - 打印 "c ..."
 - 打印 "b ... after"
 - function_b() 返回
- 打印 "a ... after"
- function_a() 返回

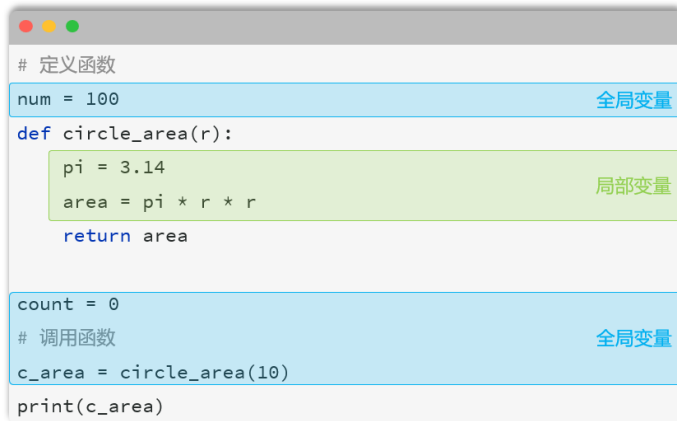
round函数：确定要输出的内容有几位小数

```
# 格式
round(3.14*2*r,1) #将内容保留一位小数
```

4.2 函数进阶

函数变量作用域

- 变量的作用域指的是变量的作用范围(标识这个变量在哪里可以使用,在哪儿不可以使用)。



```
# 定义函数
num = 100
def circle_area(r):
    pi = 3.14
    area = pi * r * r
    return area

count = 0
# 调用函数
c_area = circle_area(10)
print(c_area)
```

全局变量：在函数之外定义的变量，称之为全局变量，在整个文件中（包括函数内）都可以使用（通常定义在文件的顶部）。

局部变量：在函数内部定义的变量，称之为局部变量，只能在该函数内部使用，外部无法访问(函数执行完毕后，会自动销毁其内部局部变量)。

global 函数

- global关键字用于明确的告诉Python解释器，在函数中要使用全局变量，使得可以在函数内部修改全局变量的值。



```
num1 = 1 # 全局变量
def fun1():
    num1 = 100 # 局部变量
    print(num1)

fun1() # 100
print(num1) # 1
```



```
num1 = 1 # 全局变量
def fun1():
    global num1
    num1 = 100 # 修改全局变量num1
    print(num1)

fun1() # 100
print(num1) # 100
```

注意：在基于global声明全局变量时，要先声明，再使用。

小结：

1.什么是局部变量，全局变量？

- 在函数内部定义的变量就是局部变量，函数外声明的变量是全局变量。

2.global关键字的作用？

- 在函数内部使用，声明接下来要使用的是全局变量，语法：global xxx

3.注意事项

- 尽量避免在函数中使用全局变量，因为会使代码难以维护和调试
- 考虑使用函数参数和返回值来传递数据，而不是依赖全局变量
- global主要用在程序的状态、配置和计数器等场景中

函数参数

传参方式

位置传参

函数-传参方式

- 传参方式指的是，在调用函数时，传递实参的方式。

1. **位置参数**：调用函数时根据函数定义时的位置来传递参数。

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu("张三", 18, "男", "北京")
print(stu)
```

要求：调用函数时参数顺序与定义函数时参数顺序完全一致

多一句没有，少一句不行，用更短的时间，教会更实用的技术



关键字传参

函数-传参方式

- 传参方式指的是，在调用函数时，传递实参的方式。

2. **关键字参数**：调用函数时以函数定义时形参名称作为关键字，以 "键=值" 的形式来传递参数(不要求顺序)。

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu(name="张三", age=18, gender="男", city="北京")
print(stu)

stu2 = reg_stu(gender="男", name="王武", city="上海", age=22)
print(stu2)

stu2 = reg_stu("赵四", 28, gender="男", city="上海")
print(stu2)
```

要求：如果位置参数与关键字参数混用，关键字参数必须在位置参数之后(关键字参数之间,没有顺序要求)

多一句没有，少一句不行，用更短的时间，教会更实用的技术



对比

位置参数 vs 关键字参数

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu("张三", 18, "男", "北京")
print(stu)
```

优点: 简洁

缺点: 可读性差、易出错、维护难

场景: 参数少 (不超过3个), 且顺序自然

```
s = calc(87, 68, 92, 85)
```

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu(name="张三", age=18, gender="男", city="北京")

stu2 = reg_stu(gender="男", name="王武", city="上海", age=22)
```

优点: 可读性强、易维护和扩展

缺点: 代码繁琐

场景: 参数较多, 或易混淆的场景

```
s = calc(math=87, chinese=68, english=92, computer=85)
```

黄金法则: 半年后回头看你今天写的代码, 能否一眼看出每个参数的含义, 如果不能, 就应该使用关键字参数。

小结:

1.位置参数

- 调用函数时, 传入的实参的顺序与定义函数时形参的顺序完全一致。

2.关键字参数

- 调用函数时, 通过 "形参名=值" 的形式传递参数, 顺序没有要求。
- 如果同时存在位置参数与关键字参数, 位置参数在前, 关键字参数在后。

3.两种传参方式的适用场景

- 一切以代码结构清晰明了 (可读性)、便于维护 (维护性) 为目标。
- 如果参数比较少 (不超过3个), 可直接使用位置参数。
- 如果参数数量较多, 建议使用关键字参数。

默认参数

- 默认参数也称为缺省参数, 用于在定义函数时, 为参数提供默认值, 调用函数时, 可以不传递有默认值的参数。

```
# 定义函数
def reg_stu(name, age, gender, city='北京'):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu("张三", 18, "男")
print(stu)

stu = reg_stu("赵四", 22, "男", "深圳")
print(stu)
```

注意: 默认参数必须放在没有默认值的参数列表的后面, 一个函数在定义时是可以设置多个默认参数的。

注意: 函数调用时, 如果为默认参数传递了值, 则会修改默认的参数值; 如果没有传递该参数, 则直接使用默认值。

不定长参数

位置传参

```
# 定义函数
def calc_data(*args):

# 调用函数
data = calc_data(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
print(data)

data = calc_data(100, 200, 300, 400, 500)
print(data)
```

注意：传递的所有匹配的位置参数都会被args变量收集，这些参数会合并封装为一个元组，args是元组类型（注意并不会封装关键字参数）。

注意：args只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 *data）。

关键字传参

```
# 定义函数
def calc_data(*args, **kwargs):
    min_data = min(args)
    max_data = max(args)
    avg_data = sum(args) / len(args)

    if kwargs.get('round'):
        avg_data = round(avg_data, kwargs.get('round'))

    return min_data, max_data, avg_data

data = calc_data(100, 200, 300, 400, round=2, count=0)
print(data)

data = calc_data(33, 11, 28, 91, 32, 75, 49)
print(data)
```

注意：参数是以 "键=值" 形式传递的关键字参数，这些 "键=值" 参数都会被kwargs接受，并合并封装为一个字典类型。

注意：kwargs只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 **options）。

小结：

1. 什么是不定长参数?

- 参数个数不确定, 此时就可以使用不定长参数解决这类问题

2. 不定长参数的分类?

- `*args`: 不定长位置参数, 函数调用时, 通过位置参数传递多个参数封装到一个元组(tuple)中
- `**kwargs`: 不定长关键字参数, 函数调用时, 通过关键字参数传递多个参数封装到一个字典(dict)

3. *args与**kwargs的应用场景?

- `*args`适用于处理数量不确定的数据
- `**kwargs`适用于处理数量不确定的选项 (函数的配置参数, 用来定制函数的行为)

```
def calc_data(*args, **kwargs):
```

```
# 核心数据: 你要什么
点奶茶("珍珠奶茶")

# 选项: 你要什么样的
点奶茶("珍珠奶茶", 甜度="少糖", 冰度="去冰", 加料=["布丁", "珍珠"], 大小="大杯")
```

参数类型

- 普通参数: 数字、布尔、字符串、列表、元组、集合、字典等。
- 特殊参数: 函数。

```
def circle_area(r):
    area = 3.14 * r ** 2
    return area

area = circle_area(10)
print(area)
```

```
def calc_score(score_list):
    max_s = max(score_list)
    min_s = min(score_list)
    avg_s = round(sum(score_list) / len(score_list), 1)
    return max_s, min_s, avg_s

s_list = [589, 609, 605, 643, 677, 455, 477, 489, 503]
max_score, min_score, avg_score = calc_score(s_list)
```

```
def add(x, y):
    return x+y

def subtract(x, y):
    return x-y

def calc(x, y, oper):
    return oper(x, y)

result = calc(10, 20, add)
print(result)
```

传递的是实际要计算的数据

主要是将函数也可以作为参数进行传递

匿名函数

- 匿名函数指的是没有名称的函数, 需要通过lambda表达式来声明函数, 可以简化简单函数的编写 (单行表达式)。

```
# 定义命名函数
def 函数名(参数列表):
    函数体...

def out_line():
    print('-----')

def add(x, y):
    return x + y

out_line()
print(add(10, 20))
```

命名函数

```
# 定义匿名函数
lambda 参数列表 : 函数体

lambda : print('-----')
lambda x, y: x + y
```

匿名函数

注意：函数逻辑比较简单(单行表达式)且只在一个地方使用时，可以考虑使用匿名函数，简化书写（通常作为高阶函数的参数使用）。

注意：匿名函数中可以返回结果，也可以不返回结果。返回结果时，不需要写return，表达式的运行结果就是要返回的结果。

小结：

1.匿名函数的定义方式

```
# 定义匿名函数

lambda 参数列表 : 函数体
```

2.命名函数与匿名函数的选择？

- 建议使用匿名函数的情况：函数逻辑简单，只在一个地方调用（常作为高阶函数的参数）
- 建议使用命名函数的情况：函数逻辑复杂，需要多步操作，需要多个地方重复使用或需要加文档说明的场景

python中sort函数的用法

里边有key, reverse两种参数

key主要是决定如何排序，reverse是决定从大到小还是从小到大（False表示从小到大）

```
data_list = ["C++", "C", "Python", "Jack", "PHP", "Java"]
data_list.sort(key=lambda item: {__len__}:len(item), reverse=False)
print(data_list)
```

列表推导式与生成器表达式对比

- 列表推导式：[要插入的值 for i in 数据集 if 条件]
- 特点：立即求值，一次性生成整个列表并存入内存，占用内存空间大
- 场景：数据量不大，需要全部立即生成的场景；需要重复使用，多次访问的场景；
- 生成器表达式：(要插入的值 for i in 数据集 if 条件)
- 特点：惰性求值，按需逐个生成元素，无需同时存储所有元素，节省内存
- 场景：处理大数据集、避免一次性加载所有数据到内存；sum()/max()/min等函数的参数

4.3 类型注解

- 类型注解是Python中的一种语法特性，用于明确标识变量、函数参数和返回值的数据类型，从而使代码更清晰、更安全、更易维护。

```
# 定义变量
a = 695

score = 98.5

hobby = "Python"

flag = True

pic = None

names = ["A", "C", "E"]
phones = {"13309091111", "15209109121"}
options = {"count": 0, "total": 0}
goods = ("手机", 5999, 1)
```

```
# 定义变量
a: int = 695

score: float = 98.5

hobby: str = "Python"

flag: bool = True

pic: None = None

names: list[str] = ["A", "C", "E"]
phones: set[str] = {"13309091111", "15209109121"}
options: dict[str, int] = {"count": 0, "total": 0}
goods: tuple[str, int, int] = ("手机", 5999, 1)
```

类型推断

- 类型推断是指Python解释器自动推断出变量、表达式或函数返回值的数据类型的能力，而无需开发者显式声明。

```
# 定义变量
a = 695

score = 98.5

hobby = "Python"

flag = True

pic = None

names = ["A", "C", "E"]
phones = {"13309091111", "15209109121"}
options = {"count": 0, "total": 0}
goods = ("手机", 5999, 1)
```

```
a = 695

score = 98.5

hobby = "football"

flag = True

pic = None

names = ["张三", "李四", "王五"]
```

D:/py_code_workspace/py_project1/05. 函数进阶.py
names: list[str] = ["张三", "李四", "王五"]

```
goods = ("鼠标", "键盘", "USB")
```

注意：在对变量进行直接赋值，或者涉及到变量的运算、容器的推导等场景时，解释器会自动推导出变量的类型。

小结：

1.类型注解的写法？

- 变量: 数据类型 (如 a: int)

2.常见类型的写法

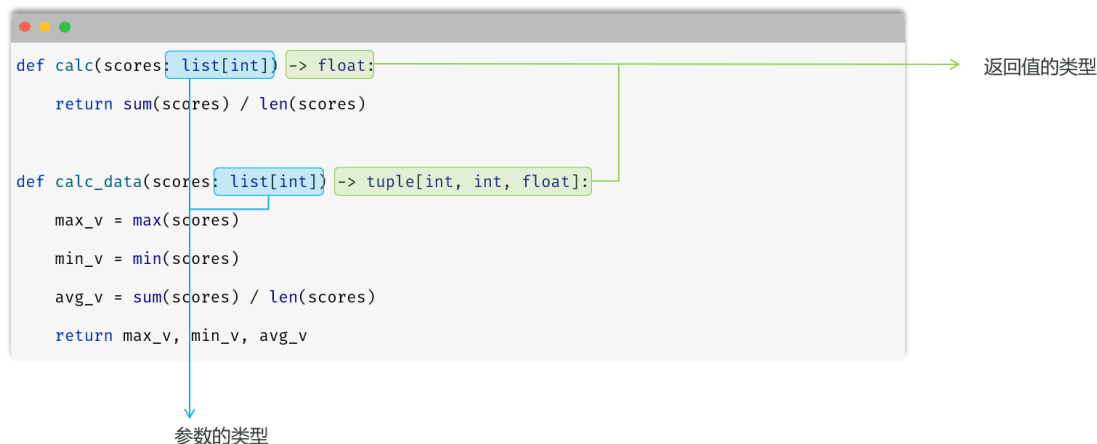
- int、float、bool、str、None、list、set、tuple、dict
- str | int

3.为什么要使用类型注解，有什么好处呢？

- 代码结构更清晰、代码逻辑更安全、易维护
- 更准确的代码自动提示
- 提前发现代码潜在问题

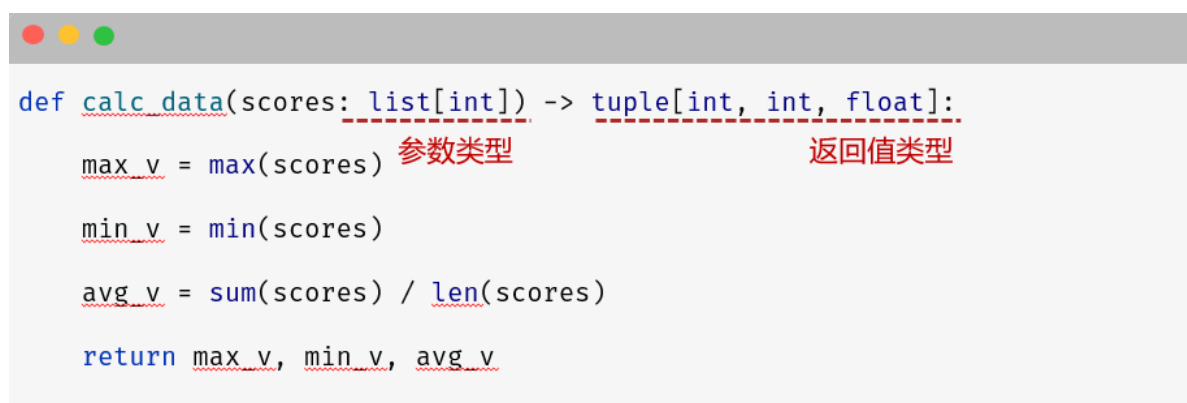
函数类型注解

- 为函数添加类型注解，其实主要就是为函数的参数和返回值添加类型注解，具体语法如下：



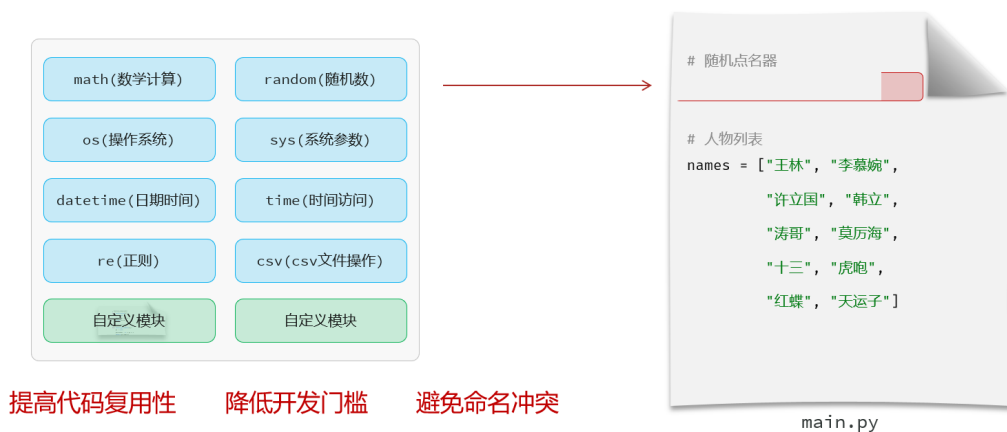
小结：

1.函数中类型注解的语法？



5. 模块

- Python模块(module)：一个.py文件就是一个模块，模块是Python程序的基本组织单位。在模块中可以定义变量、函数、类，以及可执行的代码。



5.1 导入模块

模块导入方式

- 在使用模块中提供的功能之前，必须先导入，再使用。
- 导入模块的具体语法如下：



导入形式	代码样例	调用方式	调用方式
<code>import 模块名</code>	<code>import random, os</code>	模块名.功能名	<code>random.randint(10, 100)</code>
<code>import 模块名 as 别名</code>	<code>import random as rd</code>	别名.功能名	<code>rd.randint(10, 100)</code>
<code>from 模块名 import 功能名</code>	<code>from random import randint,choice</code>	功能名	<code>randint(10, 100)</code>
<code>from 模块名 import 功能名 as 别名</code>	<code>from random import randint as rint</code>	别名	<code>rint(10, 100)</code>
<code>from 模块名 import *</code>	<code>from random import *</code>	功能名	<code>randint(10, 100)</code>

小结：

1.什么是模块? 有什么用?

- 模块：就是一个python文件(.py)，其中就包含了变量、函数、类，以及可执行的代码。
- 作用：提高代码复用性，降低开发门槛

2.导入模块的常用语法? (导入模块的语句，一般写在py文件的开头)

- `import 模块名 [as 别名]`
- `from 模块名 import 功能名 [as 别名]`
- `from 模块名 import *`

5.2 自定义模块

- 当开发一些复杂的项目，为了让项目结构更清晰，更便于项目的维护管理 及 代码的复用，可能会把一个项目拆分为若干个模块。



注意： 每一个python文件都可以作为一个模块，模块的名字就是文件的名字（建议使用python标识符定义，规范命名）。

`__all__`

__all__是一个模块级别的特殊变量，用于指定 `from 模块名 import *` 时会导入哪些功能(*通配了哪些功能)。

```
my.py
__all__ = ["log_separator1", "log_separator3", "PI"]

PI = 3.1415926
NAME = "黑马*涛哥"

def log_separator1():
    print("- " * 30)

def log_separator2():
    print("+ " * 30)

def log_separator3():
    print("# " * 30)

def log_separator4():
    print("* " * 30)
```

```
test.py
from my import *

log_separator1()
log_separator3()
log_separator2()
log_separator4()

print(PI)
print(NAME)
```

在导入模块中测试代码的写法：

```
if __name__ == "__main__":
    测试代码
```

小结：

1.name 与 __all__ 这两个特殊变量的作用是什么？

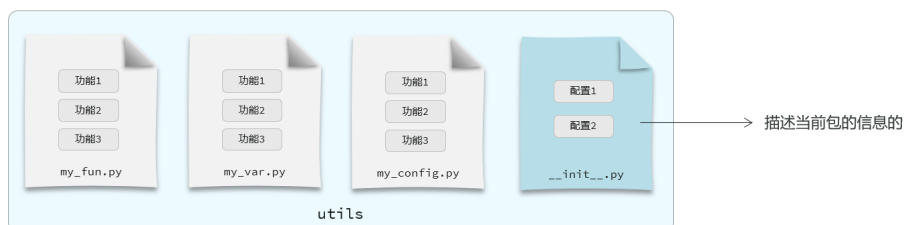
name是Python中非常重要的内置变量，表示的是当前模块的名称

- 当模块直接运行时：**name**的值为"main" (if **name** == "main")
- 当模块被导入时：**name**等于模块的文件名(不含.py后缀)

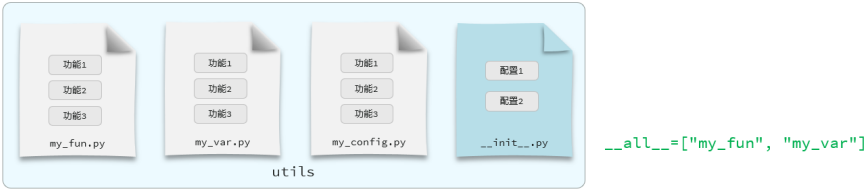
__all__：控制 `import *` 时导入哪些功能

5.3 包

- 包：本质就是一个文件夹，该文件夹中可以包含若干python模块（.py文件），文件夹下还包含了一个**__init__.py**。
- 作用：模块文件较多时，用来管理多个模块。（包的本质也是一个模块）



包的导入方式



导入形式	代码样例	调用方式	调用方式
<code>import 包名.模块名</code>	<code>import utils.my_fun</code>	包名.模块名.功能名	<code>utils.my_fun.log_separator1()</code>
<code>from 包名 import 模块名</code>	<code>from utils import my_fun</code>	模块名.功能名	<code>my_fun.log_separator1()</code>
<code>from 包名 import *</code>	<code>from utils import *</code>	模块名.功能名	<code>my_fun.log_separator1()</code>
<code>from 包名.模块名 import 功能名</code>	<code>from utils.my_fun import log_separator1</code>	功能名	<code>log_separator1()</code>
<code>from 包名.模块名 import *</code>	<code>from utils.my_fun import *</code>	功能名	<code>log_separator1()</code>

注意：在通过 `'from 包名 import *'` 导入全部模块的时候，需要在 `__init__.py` 文件中添加 `'__all__=[]'`，控制允许导入的模块列表。

小结：

1.什么是包? 有什么作用?

•包就是一个文件夹，里面可以存储很多Python模块（py文件），通过包可以对模块进行归类

2.init.py文件的作用?

- 标识这是一个包，而不是普通的文件夹
- 控制在 `import *` 时导入的模块列表(all变量)

3.导入包的方式?

- `import 包名.模块名`
- `from 包名 import 模块名`
- `from 包名 import *`
- `from 包名.模块名 import 功能名`
- `from 包名.模块名 import *`

6. 面向对象

6.1 基础

- 类：描述的是一组具有相同属性（特征）和方法（功能/行为）的模板。



- 对象：对象是类的实例，是基于类创建出来的（实例对象）。



提示：对象是由类创建出来的，创建对象的过程，也称为对象的实例化。一个类可以创建无数个对象。

6.1.1 类与对象

类的定义

- 定义类的语法如下：

```
# 定义类
class 类名:
    pass

# 创建对象
对象名 = 类名()
对象名.属性名1 = 属性值1
对象名.属性名2 = 属性值2
```

```
# 定义类
class Car:
    pass

# 创建对象
c1 = Car()
c1.brand = "BMW"
c1.name = "X5"
c1.price = 500000
print(c1.__dict__)
```

说明：类名的命名规范，遵循大驼峰命名法，每个单词的首字母都是大写，单词之间没有分隔符，比如：UserInfo，UserAccount。

说明：dict是Python中用户自定义类实例的一个特殊属性，用于以字典形式存储对象的属性。

更推荐的定义方法：

- 定义类的语法如下：

```
# 定义类
class 类名:
    def __init__(self, 参数列表):
        self.属性名 = 参数值
        self.属性名 = 参数值

# 创建对象
对象名 = 类名(参数列表)
```

```
# 定义类
class Car:
    def __init__(self, c_brand, c_name, c_price):
        self.brand = c_brand
        self.name = c_name
        self.price = c_price

# 创建对象
c1 = Car("BMW", "X5", 500000)
print(c1.__dict__)
```

推荐

self: 方法的第一个参数，表示当前创建的实例对象

__init__: 初始化方法，对象创建后自动调用，主要用于设置对象的初始状态（设置对象属性）

说明：定义在类的外面的称之为函数，定义在类中的函数称之为方法。

小结：

1.定义类时，类名的命名规范？

•大驼峰命名法，UserInfo、UserAccount

2.定义类时，init方法的作用？self参数的作用？

- init 是初始化方式，对象创建时自动调用，主要用于设置对象的初始状态（设置对象属性）
- self是类中定义的方法的第一个参数，表示当前创建的实例对象

6.1.2 实例方法

- 在类中定义实例方法时，定义语法与之前学习的函数定义的方式是一致的。

```
# 定义类
class 类名:
    def __init__(self, 形参列表):
        self.属性名 = 参数值
        self.属性名 = 参数值

    def 方法名(self, 形参列表):
        ...
    def 方法名(self, 形参列表):
        ...

# 创建对象
对象名 = 类名(参数列表)
对象名.方法名(实参)
```

```
class Car:
    def __init__(self, brand, name, price):
        self.brand = brand
        self.name = name
        self.price = price

    def running(self):
        print(f"{self.brand} {self.name} 正在高速行驶...")
    def total_cost(self, discount, rate):
        return self.price * discount + self.price * rate

c1 = Car("BMW", "X5", 500000)
total_cost = c1.total_cost(0.9, 0.1)
print(f"提车总价为: {total_price:.0f}")

c1.running()
```

self表示当前实例对象，方法调用时无需传递

6.1.3 魔法方法

- 魔法方法是指Python中提供的以双下划线开头和结尾的特殊方法，用于定义类的特殊行为，比如：__init__。
- 魔法方法是不需要我们手动调用的，Python会在合适的时机自动调用。

魔法方法	描述
<code>__init__</code>	初始化方法
<code>__str__</code>	字符串表示的方法
<code>__eq__</code>	比较两个对象是否相等（equal）
<code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>	支持比较两个对象的大小(小于(less than), 小于等于(less than or equal), 大于(greater than), 大于等于(greater than or equal))

```
class Car:
    def __init__(self, brand, name, price):
        self.brand = brand
        self.name = name
        self.price = price
    def running(self):
        print(f"{self.brand} {self.name} 正在高速行驶...")
    def __str__(self):
        return f"{self.brand} {self.name} {self.price}"
    def __eq__(self, other):
        return self.price == other.price and self.brand == other.brand and self.name == other.name
    def __lt__(self, other):
        return self.price < other.price

c1 = Car("BMW", "X5", 500000)
print(c1)      BMW X5 500000
c2 = Car("BMW", "X5", 500000)
print(c2)      BMW X5 500000

print(c1 == c2)    True
print(c1 < c2)     False
```

小结:

1. 什么是魔法方法?

- Python中提供的__xxx__形式的特殊方法
- 魔法方法无需手动调用, Python会在合适的时机自动调用

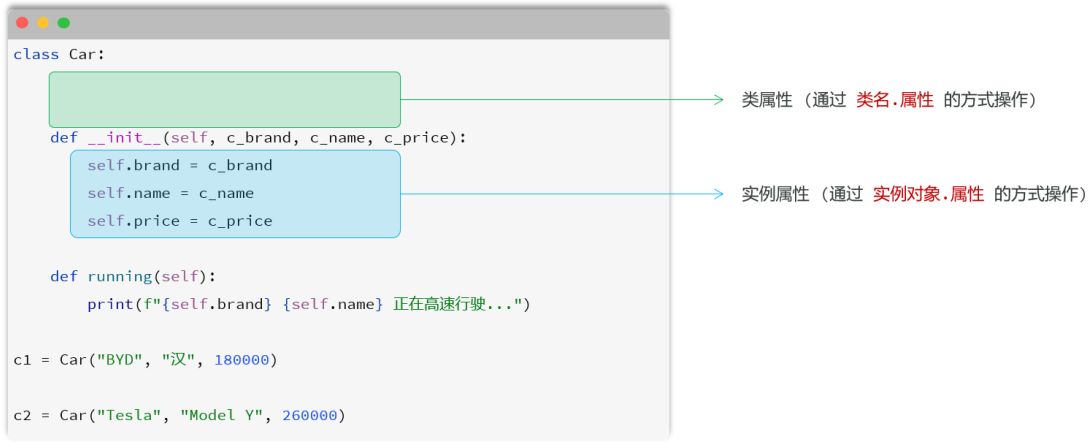
2. 常用的魔法方法有哪些, 作用是什么?

- __init__
- __str__
- __eq__
- __lt__ , __le__ , __gt__ , __ge__

6.1.4 实例属性与类属性

- 属性分为：

- 实例属性：实例属性属于每个具体对象的属性，每个对象都是独立的。（各个对象特有的数据）
- 类属性：类属性是属于类本身的属性，所有实例共享的。（所有对象共享的数据或配置）



6.2 高级

6.2.1 封装



封装

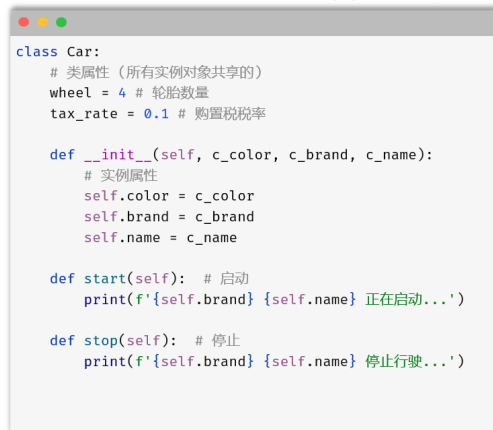
多一句没有，少一句不行，用更短的时间，教会更实用的技术



私有属性、私有方法

- 封装就是把数据（属性）和操作数据的函数（方法）捆绑在一起，形成一个独立的单元（类），并隐藏内部的实现细节，只对外暴露必要的功能（方法）

公共属性、公共方法



私有：私有的属性和方法只能在类的内部使用; Python中并没有真正的私有机制，约定在私有属性名和方法名前加__（两个下划线）。

小结：

1.简单描述一下什么是封装？

- 封装就是将数据（属性）及操作数据的方法绑定在一起，形成一个独立的类。

2.私有属性、方法与公共的有什么区别呢？该怎么定义？

- 私有的只能在类的内部访问，在外部是无法访问的；而公共的属性、方法在类的内部、外部都是可以访问的。

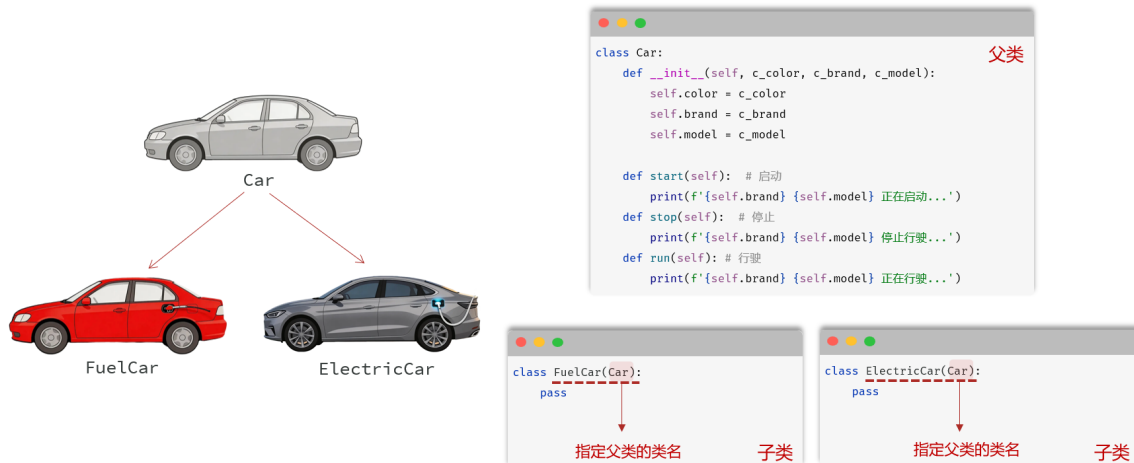
- 私有的属性和方法在命名的时候，均以__作为开头。（Python中没有真正的私有机制）

3.私有属性、方法的应用场景？

- 如果某些属性和方法是不对使用者公开的，就可以将其设置为私有的。

6.2.2 继承

- 继承描述的是两个类之间的关系，子类继承父类，就可以获取到父类的属性和方法（非私有）



小结:

1. 继承的语法格式?

class 子类(父类):

pass

2. 子类继承父类，是将父类所有的属性和方法都继承下来了吗?

• 没有

• 子类继承父类，会将父类公共的属性和方法继承下来（不含私有）

重写

- 重写**是指子类继承父类后，如果父类中的方法不满足需求，可以在子类中重新定义父类中已有的方法（方法名相同），从而用子类的实现替换父类的实现。



小结:

1. 什么是方法重写? 重写的语法?

• 在继承体系中，如果父类的方法不能满足需求，就可以在子类中重新定义一个与父类同名的方法，从而用子类自己的实现来替换父类的实现，这个就称之为重写

2. 在子类中，如何调用父类中的方法?

•方式一: super().方法名()

•方式二: 父类名.方法名(self)

多继承

- 多继承指的是一个子类,同时继承了多个父类的情况(会将多个父类中的非私有的属性和方法都继承下来)。
- 语法:

```
class 子类名(父类名1, 父类名2, 父类名3, ...):  
    代码  
    ...
```

注意: 当一个类继承了多个父类时,默认优先使用第一个父类中的同名属性或方法,可以使用 类名.mro 属性 或 类名.mro() 方法查看调用顺序。

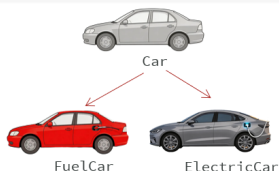
MRO : Method Resolution Order(方法解析顺序)

6.2.3 多态

- 多态是指同一个方法,具有不同的形态、行为、表现。

```
class Car:  
    ...  
    def charge(self):  
        print('正在补充燃料...')  
  
class FuelCar(Car):  
    def charge(self):  
        print('燃油汽车正在加油 ...')  
  
class ElectricCar(Car):  
    def charge(self):  
        print('电动汽车正在充电...')
```

```
def handle_charge(car: Car):  
    car.charge()  # 参数类型为父类,支持传入任意子类  
  
handle_charge(FuelCar("BMW", "X5", "白色", 500, "张三"))  
handle_charge(ElectricCar("BYD", "汉", "白色", 500, "张三"))  
  
# 同一调用语句,传入对象不同,触发不同行为
```



•如: 定义函数时, 参数类型指定为父类类型, 在执行的时候传入不同的子类对象, 就具有不同的形态
鸭子类型

- **鸭子类型(Duck Typing)**是指如果它走路像鸭子,叫起来叫鸭子,那么它就是鸭子。
- 在鸭子类型中,我们**关注**的是对象的**行为**(它有什么方法),而不是对象的**类型**(它是什么类)。

```
class Dog:  
    ...  
    def swimming(self):  
        print(f'{self.age} 岁的 {self.name} 正在游泳...')  
  
class Duck:  
    ...  
    def swimming(self):  
        print(f'{self.age} 岁的 {self.name} 正在游泳...')  
  
class Pig:  
    ...  
    def swimming(self):  
        print(f'{self.age} 岁的 {self.name} 正在游泳...')
```

```
def go_swimming(duck):  
    duck.swimming()  
  
# 测试代码  
if __name__ == '__main__':  
    go_swimming(Dog("旺财", 4))  
    go_swimming(Duck("唐老鸭", 2))  
    go_swimming(Pig("佩奇", 1))
```

提示：鸭子类型的优势是不需要存在继承关系，只要对象有相应的方法就能使用。

小结：

1.什么是多态？

- 多态是指同一个方法，具有不同的表现形态

- 如：定义函数时，参数类型指定为父类类型，在执行的时候传入不同的子类对象，就具有不同的形态

2.什么是鸭子类型？

- 鸭子类型也是多态的一种体现，鸭子类型关注的是对象的行为（有什么方法），而不是对象的类型（无需继承关系）

6.2.4 抽象类

```
# 抽象类：是一种只能被继承，不能被直接实例化的类，作用就是规定子类必须要实现哪些方法，强制子类必须遵守统一的代码规范
# Python中的抽象类，需要继承 abc 模块中的ABC类 ---> ABC: Abstract Base Class
```

```
from abc import ABC, abstractmethod
```

```
# 获取会员最大借阅数量(需要在子类中实现)
```

```
@abstractmethod 1 个用法
```

```
def get_max_books(self) -> int:
```

```
    pass
```

7.异常

- 异常（也称为Bug）就是程序运行过程中出现的错误，它会中断程序的正常执行流程。
- 作用：
 - 保证数据、逻辑的正确性，避免程序执行混乱
 - 在开发阶段，尽量发现更多的问题，尽早解决问题，保障程序正常执行

`NameError`

`TypeError`

`IndexError`

`KeyError`

`ValueError`

使用方法

- 程序运行过程中出现异常，有两种处理方案：

1. 不做处理：整个程序因为一个Bug，中断执行。(之前编程的程序)
2. 捕获异常：按照我们自己的处理方式，处理完异常，程序继续执行。(编写程序时，做好预案，出现异常，按预案处理)

```
try:
    可能出现异常的业务代码1
    可能出现异常的业务代码2
    ...
except [异常类型 as 变量名]:
    出现异常时的预案
[finally:
    不管是否出现异常，都会执行的代码 ]
```

```
try:
    print("=====")
    print(my_name)
    print("=====")
except NameError as e:
    print("程序运行报错，错误信息：", e)
finally:
    print("释放资源 ~")
```

小结：

1.为什么要捕获异常？

- 当程序运行出现异常，提供预案，处理异常，而不是让其中止程序运行

2.如何捕获异常，具体的语法？

```
try:
    print("ABC".hello)
except NameError as e:
    print("名称不存在，请检查，具体信息：", e)
except ZeroDivisionError as e:
    print("0不能做被除数，请检查，具体信息：", e)
except IndexError as e:
    print("索引错误，请检查，具体信息：", e)
except Exception as e:
    print("其他错误，请检查，具体信息：", e)
finally:
    print("无论正常执行还是出现异常，都要释放资源 ~")
```

异常的传递

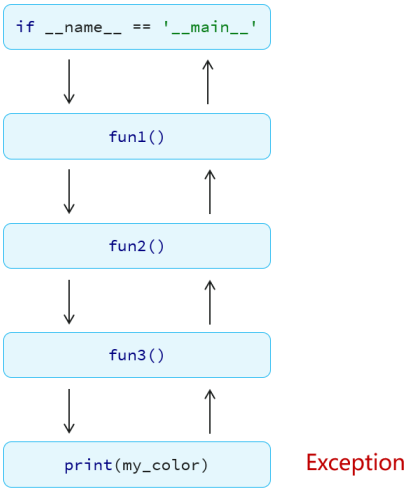
- 异常传递就是异常在函数调用中层层上报的过程，直到有人处理它，或者程序崩溃。

```
def fun1():
    print("fun1 ... running ...")
    fun2()

def fun2():
    print("fun2 ... running ...")
    fun3()

def fun3():
    print("fun3 ... running ...")
    print(my_color)

if __name__ == '__main__':
    fun1()
```



二、AI应用

1.网络基础知识

HTTP协议-请求数据格式

```
POST /api/courses HTTP/1.1
Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-US;q=0.7,en-GB;q=0.6
Connection: keep-alive
Content-Length: 139
Content-Type: application/json
Host: localhost:90
Origin: http://localhost:90
Referer: http://localhost:90/resource/course
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/143.0.0.0

{"phone":"18808088080","channel":1,"name":"虎跑","gender":1,"age":"32"}
```

请求行(请求方式、资源路径、协议)

请求头(格式key: value)

请求体(请求参数部分，GET方式没有，POST可以有)

- 请求方式：
- GET：请求参数在请求行中，没有请求体。如： /api/courses?name=Python&status=1 。GET请求请求参数大小在浏览器中是有限制的。
 - POST：请求参数在请求体中，POST请求大小是有限制的。

HTTP协议-响应数据格式

```
HTTP/1.1 200
Server: nginx/1.24.0
Date: Tue, 16 Dec 2025 12:38:08 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive

{code: 1, msg: "success", data: null}
```

响应行(协议、状态码)

响应头(格式key: value)

响应体(存放服务端响应的数据)

- 200：客户端请求成功
- 400：请求参数错误
- 404：请求资源不存在，url输入有误，或者网站资源被删除了
- 500：服务器发生了不可预期的错误

小结：

1.HTTP协议中请求及响应数据的数据格式？

- 请求格式：请求行(请求方式、资源路径)、请求头(key:value)、请求体(post方式)
- 响应格式：响应行(状态码)、响应头(key:value)、响应体

2.Apifox调用DeepSeek

Json格式

JSON(JavaScript Object Notation)是前端的一种对象表示方法。表示形式类似于Python中的字典，都是key:value这种形式，不过所有的key都必须使用双引号引起来，值可以是任何类型：

- **对象**：用{}表示，{}之间是键值对形式，键是字符串，值可以是任意其它类型
- **数字**：整数和小数都是数字，例如：12、3.14
- **字符串**：用""引起来，例如："jack"
- **布尔**：有两种值：true或false
- **列表**：用[]表示，[]中是列表的元素，多个元素以,分割。

```
{
  "model": "deepseek-chat",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": false
}
```

Apifox调用DeepSeek

DeepSeek API 文档

快速开始

首次调用 API

模型 & 价格

Temperature 设置

Token 用量计算

概述

错误码

新闻

DeepSeek-V3.2 正式版发布 2025/12/01

DeepSeek-V3.2-Exp 发布 2025/09/29

DeepSeek-V3.1 更新 2025/09/22

DeepSeek-V3.1 发布 2025/08/21

DeepSeek-R1-0528 发布 2025/05/28

DeepSeek-V3-0324 发布 2025/03/25

DeepSeek-R1 发布 2025/01/20

DeepSeek APP 发布 2025/01/15

DeepSeek-V3 发布 2024/12/26

DeepSeek-V2.5-1210 发布 2024/12/10

DeepSeek-R1-Lite 发布 2024/11/20

DeepSeek-V2.5 发布 2024/09/05

API 上线额度缓存 2024/08/02

API 升级新功能 2024/07/25

首次调用 API

DeepSeek API 使用与 OpenAI 兼容的 API 格式，通过修改配置，您可以使用 OpenAI SDK 来访问 DeepSeek API，或使用与 OpenAI API 兼容的软件。

PARAM	VALUE
base_url	https://api.deepseek.com
api_key	apply for an API key

* 出于与 OpenAI 兼容考虑，您也可以将 base_url 设置为 https://api.deepseek.com/v1 来使用，但注意，此处 v1 与模型版本无关。

* deepseek-chat 和 deepseek-reasoner 都已经升级为 DeepSeek-V3.2，deepseek-chat 对应 DeepSeek V3.2 的 **非思考模式**，deepseek-reasoner 对应 DeepSeek V3.2 的 **思考模式**。

调用对话 API

在创建 API key 之后，您可以使用以下样例脚本的来访问 DeepSeek API，样例为非流式输出，您可以将 stream 设置为 true 来使用流式输出。

curl python nodejs

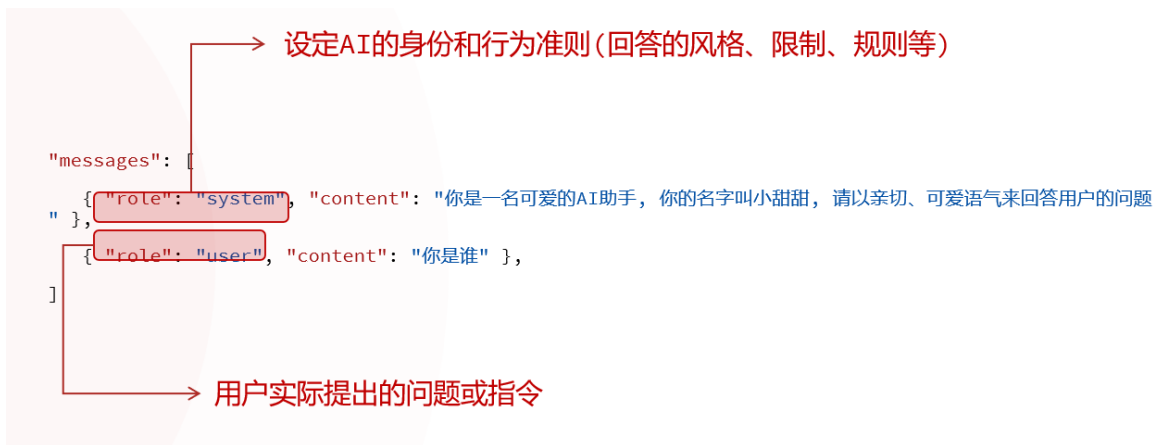
url地址

请求头(key: value)

请求体(json格式)

```
curl https://api.deepseek.com/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $[DEEPSEEK_API_KEY]" \
-d '{
  "model": "deepseek-chat",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": false
}'
```

小结：



3. 会话记忆

会话记忆-处理方案(会话历史滚雪球)



小结:



4. 安装第三方库

如何安装第三方的软件包?

•安装软件包（最新版本）：pip install openai

- 安装软件包（指定版本）： `pip install openai=2.13.0`
- 卸载软件包： `pip uninstall openai`
- 列出已安装的包： `pip list`
- 查看包详情： `pip show openai`

5. 提示词工程

提示词(Prompt)：是引导大模型(LLM)进行内容生成的命令（一句话、一个问题等）。

提示词工程(Prompt Engineering)：通过有技巧的编写提示词，使大模型生成出尽可能符合预期的内容，这一持续性的过程就称为提示词工程。

提示词工程

- 01 给大模型设定角色与能力
- 02 明确核心请求与任务
- 03 按步骤拆解复杂任务
- 04 指定风格与语气
- 05 明确要求输出格式
- 06 提供输入输出的示例

你是一名经验丰富的历史老师，擅长用生动有趣的方式讲述复杂的历史事件。现在，请完成以下任务：

1. 核心任务： 向一位高中生解释法国大革命爆发的主要原因。
2. 表达要求：
语气与风格： 使用简洁、口语化、充满课堂热情的语气，避免学术黑话。整体叙事可以借鉴《人类群星闪耀时》中对历史关键时刻的描写手法，富有画面感和戏剧性。
内容禁忌： 聚焦于原因分析，不要罗列冗长的日期和事件过程表。
3. 内容与结构要求：
开头： 先用一段话，生动描述革命前法国社会的总体氛围和紧张感（即背景概述）。
主体： 分别从政治、经济、思想三个层面阐述直接原因。每个层面提炼2个关键点，并为每个关键点配1个具体、有说服力的史实例子。
结尾： 提供一个帮助学生记忆的妙招。
4. 输出格式： 请严格、完整地按照以下框架组织你的回答：
【历史现场氛围】
(在这里写你的背景概述)
【危机根源解析】
政治层面：
- 关键点1: ... 例子: ...
- 关键点2: ... 例子: ...
经济层面：
- 关键点1: ... 例子: ...
- 关键点2: ... 例子: ...
思想层面：
- 关键点1: ... 例子: ...
- 关键点2: ... 例子: ...
【记忆法宝】
(请提供一个巧妙的比喻、口诀或联想图像，将这三大层面串联起来，方便学生瞬间记忆)

6. streamlit

Streamlit是一个开源的Python库，专为数据工程师及机器学习工程师设计，用来快速基于Python代码构建交互式的web网站（无需掌握前端技术）。

官方网站： <https://streamlit.io>

使用步骤：

1. 安装streamlit: `pip install streamlit`
2. 在python文件中引入streamlit模块
3. 基于streamlit中提供的API来构建Web应用
4. 运行程序: `streamlit run xxxx.py`

基本配置：

```
st.set_page_config(
    page_title="Ex-stream-ly Cool App", # 设置页面标题
    page_icon="📦", # 设置页面的图标
    layout="wide", # 页面格式，居中还是占满
    initial_sidebar_state="expanded",
    # 添加菜单栏
    menu_items={
        'Get Help': 'https://www.extremelycoolapp.com/help',
        'Report a bug': 'https://www.extremelycoolapp.com/bug',
        'About': "# This is a header. This is an *extremely* cool app!"
    }
)
```

7. 文件操作

1. 文件操作入门

- 日常我们操作文件时，基本分为三步操作：打开、读/写、关闭。

```
# 1. 打开文件
f = open("resources/望庐山瀑布.txt", "r", encoding="utf-8")

# 2. 读取文件
content = f.read()
print(content)

# 3. 关闭文件
f.close()
```

```
# 1. 打开文件
f = open("resources/静夜思.txt", "w", encoding="utf-8")

# 2. 写入文件
f.write("窗前明月光，\n")
f.write("疑是地上霜，\n")
f.write("举头望明月，\n")
f.write("低头思故乡，\n")

# 3. 关闭文件
f.close()
```

编码：是将字符(文字、数字、符号)转换为计算机能够存储和处理的数字代码的规则系统，如：ASCII、GBK、UTF-8。

注意：如果操作完文件，并未调用close方法关闭文件，同时程序没有停止运行，那么这个文件将一直被Python程序占用，无法操作。

小结：

1. 文件的操作（读/写）分为那几步？

- 打开：open()函数
- 读/写：read()方法、write()方法
- 关闭：close()方法

2. 资源释放

- 资源释放方案：

```
# 1. 打开文件
f = open("resources/静夜思.txt", "w", encoding="utf-8")

try:
    # 2. 写入文件
    f.write("静夜思(李白)\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜。 \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡。 \n")
finally:
    # 3. 关闭文件
    f.close()
```

繁琐

```
with open("resources/静夜思.txt", "w", encoding="utf-8") as f:
    f.write("静夜思(李白)\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜。 \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡。 \n")
```

推荐

提示： with语句(上下文管理器)的核心作用就是确保资源的总是被正确获取和释放（即使发生异常，也会被正确释放），也是项目开发中的推荐方式。

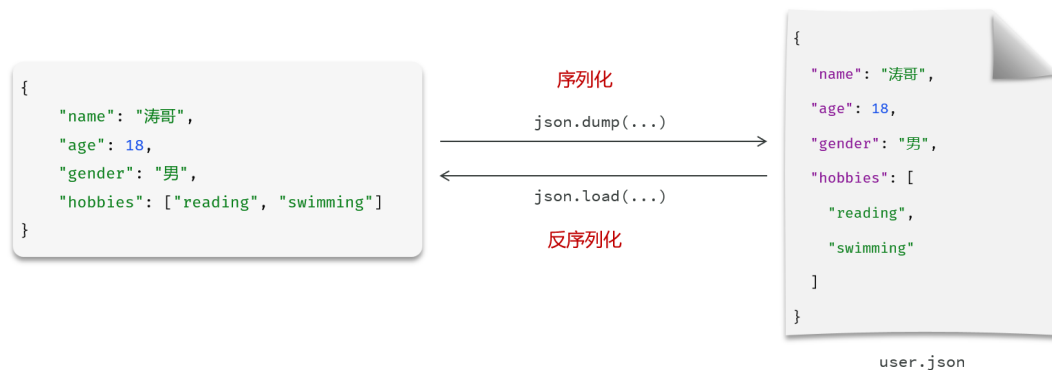
小结：

1.操作文件时，文件资源释放的最佳实践？

- try...finally
- with open()方法 (推荐，最佳实践)

3.读写json格式文件

- JSON是软件开发中最常用的数据交换格式，而为了简化JSON数据的处理，在Python标准库中就提供了处理JSON数据的核心模块 json。



- JSON是软件开发中最常用的数据交换格式，而为了简化JSON数据的处理，在Python标准库中就提供了处理JSON数据的核心模块 json。



小结：

json模块中进行对象序列化及反序列化的方法？

- `dump()`：将python对象序列化为json格式字符串并写入文件
- `load()`：从文件中读取json格式数据，并将其反序列化为python对象

```
# ensure_ascii: 默认为True，确保所有的数据输出的数据都是ascii编码(非ASCII码会进行转义); False，非ASCII码保留原样输出
# indent: 会在输出的json数据中添加缩进(格式化)
json.dump(user, f, ensure_ascii=False, indent=2)
```

4.三元运算符

● 三元运算符的写法？

- 语法: `<true_value> if 条件表达式 else <false_value>`

5.文件操作

```
# 读文件
with open("resources/望庐山瀑布.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# 写文件
with open("resources/静夜思.txt", "w", encoding="utf-8") as f:
    f.write("静夜思(李白)\n\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜. \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡. \n")
```

路径写法：

- 相对路径：相对于当前工作目录的路径；
- 绝对路径：是从文件系统的目录开始，完整的描述文件位置的路径；

操作模式：

- r：只读的方式打开文件，指针放在文件的开头。（默认模式）
- w：只写入模式，从头编辑，原有内容会被删除；文件不存在则创建新文件
- a：追加模式，新内容会被追加在原有内容之后；文件不存在则创建新文件

提示：在项目开发中，推荐使用相对路径写法，可移植性更好、路径简洁、易于阅读。

路径写法:

相对路径: 从当前文件所在目录开始查找 (推荐)

. : 当前目录 ----> ./resources/望庐山瀑布.txt ./ 可以省略

..: 上一级目录 ----> ../第2章/file/寻隐者不遇.txt ----> ../../第2章/file/寻隐者不遇.txt

绝对路径: 从文件系统根目录开始查找, 文件位置的完整路径 (注意: 反斜杠 在字符串中表示的是转义字符, \n \t)

方式一: D:\\Python-Project\\py_project01\\第3章\\resources\\望庐山瀑布.txt

方式二: D:/Python-Project/py_project01/第3章/resources/望庐山瀑布.txt

"""

三、网络机器人(爬虫)

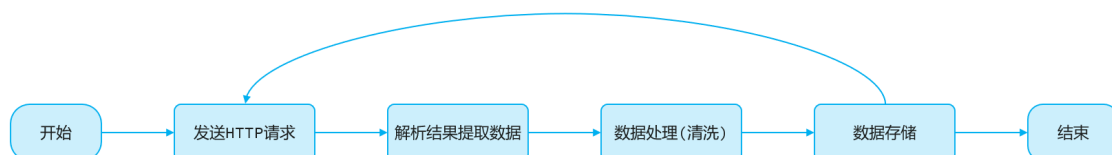
- **爬虫**: 也称为**网络爬虫**(**网络机器人**), 是一种按照一定的预设规则, 自动浏览并抓取网络数据的程序或脚本。



- 搜索引擎(百度、Google)
- 舆情监控
- 商业分析(电商比价系统)
- AI大模型训练语料
-

1.工作流程

- **网络机器人**: 也称为**网络爬虫**, 是一种按照一定的预设规则, 自动浏览并抓取网络数据的程序或脚本。



robots协议

2.robots协议

- robots协议也称为爬虫协议、爬虫规则，是指网站根目录下存放的一份文本文件robots.txt，用于告诉爬虫哪些页面可以抓取，哪些页面不能抓取。（君子协议）

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php
Sitemap: https://www.tiobe.com/sitemap_index.xml
Crawl-delay: 5
```

```
User-agent: Wandoujia Spider
Disallow: /
```

```
User-agent: Mediapartners-Google
Disallow: /subject_search
Disallow: /amazon_search
Disallow: /search
Disallow: /group/search
```

通用规则(针对于所有爬虫)

特定规则(针对于豌豆荚爬虫)

特定规则(针对于谷歌广告爬虫)

User-Agent: 用户代理，通过该请求头确认爬虫的类型

Disallow: 禁止访问的资源

Allow: 允许访问的资源

Sitemap: 网站地图，帮助爬虫更高效地获取网站内容

Crawl-delay: 爬取间隔时间，避免频繁访问造成网站压力过大

robots.txt

请务必遵守robots君子协定

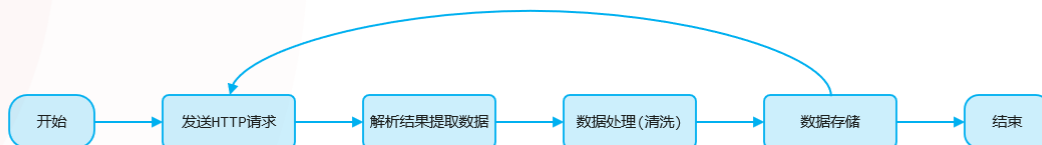
小结：

robots.txt 君子协定的数据获取规则

- User-Agent：用户代理，通过该请求头的标识，确认爬虫的身份
- Disallow：不允许访问的资源
- Allow：允许访问的资源
- Sitemap：网站地图
- Crawl-delay：访问的间隔时间

3.requests库

- Requests库的作用是什么？
 - Requests库是Python中最流行、最优雅的HTTP客户端库，让Python代码发送HTTP请求变得极其简单



使用方式：

```
import requests
target_url = "https://www.tiobe.com/tiobe-index/"
response = requests.get(target_url)
print(response.text)
```

4.网页基础知识

- 一个网页是由三个部分组成的，分别是：HTML、CSS、JS (JavaScript) 。
 - HTML：超文本标记语言，由一堆预设的标签(<h1>一级标题</h1>)构成。HTML负责网页的结构（页面元素和内容）
 - CSS：层叠样式表。CSS负责网页的表现（页面元素的外观、位置等样式，如颜色、大小等）
 - JS：全称为JavaScript，简称JS。负责网页的行为（交互效果）

5.xpath语法

- lxml：是一个高性能的HTML/XML文档的解析库，支持基于XPath语法来解析和获取网页数据。
- 安装：pip install lxml

```
from lxml import html

# 加载html文件
with open("resources/仙逆人物志.html", "r", encoding="utf-8") as f:
    html_content = f.read()

# 解析网页内容
doc = html.fromstring(html_content)
th_list = doc.xpath("//table/thead/tr[1]/th/text()")
print(th_list)

td_list = doc.xpath("//table/tbody/tr[1]/td/text()")
print(td_list)
```

Xpath：是一种用于在HTML/XML文档中导航或定位元素的查询语言，让你能够准确的定位文档中的特定元素、属性或文本。

- **Xpath**：一种在HTML/XML文档中导航或定位元素的查询语言，让你能够准确的定位文档中的特定元素、属性或文本。

表达式	描述	样例	
/	从根节点的直接子元素	/html/body/div/h1	<pre><html lang="zh-CN"> <head> <meta charset="UTF-8"> <title>仙逆人物志 - 修真世界</title> </head> <body> <div> <h1>Python</h1> <p>一门简介、快速、易用的编程语言。</p> <p>人生苦短，我用Python。</p> <p color="red">AI大模型开发、AI智能应用开发。</p> 黑马程序员 </div> </body> </html></pre>
//	从任意位置选择节点	//h1	
.	当前节点下查找	./a 与 ../a	
[n]	选择第n个元素	//p[2]	
[last()]	选择最后一个元素	//p[last()]	
[@attr]	选择有该属性的元素	//p[@color]	
[@attr='value']	选择该属性值等于指定值的元素	//p[@color='red']	
*	匹配任何元素节点	//body/div/*	
@*	匹配元素的任何属性	//body/div/a/@*	
text()	获取文本内容	//div/p/text()	

6.csv

- **CSV**: (**C**omma-**S**eparated **V**alues, 逗号分隔值), 是一种简单、通用的文本文件格式, 用于存储表格数据, 可以直接使用Excel打开。

方式一

```
with open("resources/01.csv", "w", encoding="utf-8") as f:
    f.write("姓名,年龄,性别,爱好\n")
    f.write("王林,29,男,'Python,Java'\n")
    f.write("红蝶,18,女,Java\n")
```

方式二

```
import csv
with open("resources/02.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["姓名", "年龄", "性别", "爱好"])
    writer.writeheader()
    writer.writerow({"姓名": "王林", "年龄": 29, "性别": "男", "爱好": "Python,Java"})
    writer.writerow({"姓名": "红蝶", "年龄": 18, "性别": "女", "爱好": "Java"})
```

7.正则表达式

- **正则表达式** (**R**egular **E**xpression) 是一种用特定语法规则组成的字符串模式, 用来描述、匹配或替换文本中符合某种规则的字符序列, 可以理解为是专门用于文本处理的“高级查找和匹配公式”。

1[3-9]\d{9}

18809091231是我的手机号, 你记住了吗? 我的另一个手机号是18800001266, QQ号是1779989922你记住了吗?

正则表达式操作

```
import re

s = "我的手机号是18809091231你记住了吗? 我的另一个手机号是18800001266, QQ号是1779989922你记住了吗?"

result = re.match(r"1[3-9]\d{9}", s) # match - 从字符串的开头开始匹配 (返回Match对象)
print(result.group())

result = re.search(r"1[3-9]\d{9}", s) # search - 从任意位置开始, 搜索第一个匹配项 (返回Match对象)
print(result.group())

result = re.findall(r"1[3-9]\d{9}", s) # findall - 从任意位置开始, 搜索所有匹配项 (返回Match对象list)
print(result)
```

小结:

什么是正则表达式, 有什么用?

• 定义的是字符串的组成规则, 根据规则匹配或替换文本中符合这个规则的字符串

字符串前的r标识什么意思?

• r表示当前这个字符串中的转义字符无效, 作为普通字符串使用

re模块提供的如下三个函数的作用与区别?

• match("正则表达式", "文本字符串"): 从字符串的开头开始匹配

• search("正则表达式", "文本字符串"): 从任意位置开始, 搜索第一个匹配项

• findall("正则表达式", "文本字符串"): 从任意位置开始, 搜索所有匹配项

语法:

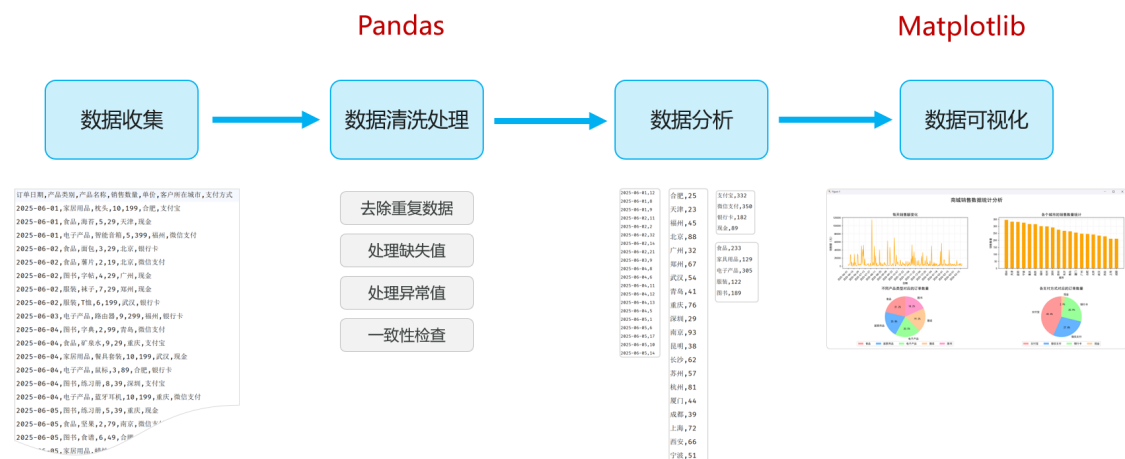
- 正则表达式的写法多种多样，具体的语法如下：

表达式	描述	表达式	描述
普通字符	字母、数字、汉字及大多数字符，直接匹配自身	*	出现任意次（0次或无数次）
.	匹配任意一个字符(除\n)	+	至少出现1次（1次或无数次）
\d	匹配数字 0-9	?	至多出现一次（0次或1次）
\D	匹配非数字	{m}	出现m次
\w	匹配单词字符，即a-z、A-Z、0-9、_	{m,}	至少出现m次
\W	匹配非单词字符	{m,n}	出现m到n次
\s	匹配空白字符，即 空格、\t		或的意思，匹配左右任意一个表达式
\S	匹配非空白字符	()	将括号中的字符作为一个分组
		^	匹配字符串开头
		\$	匹配字符串结尾

四、数据分析

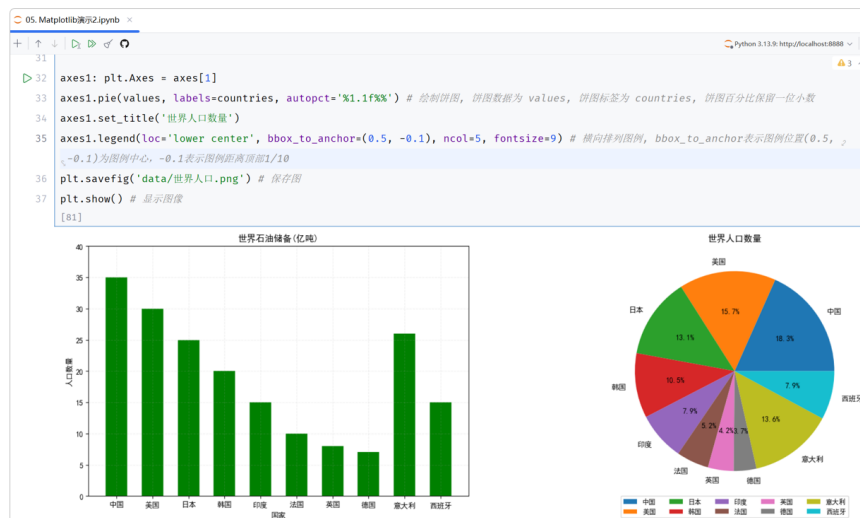
数据分析

- 数据分析**：从一堆看似杂乱的数据中，通过数据清洗、分析、可视化等手段，找出有价值的信息和结论，从而帮我们解决实际问题（如：用户订单数据的分析、电影榜单数据分析、学校学生成绩分析等）。



1.Jupyter Notebook

- Jupyter Notebook**是一个基于Web网页的、交互式的编程笔记本，让你可以把代码、运行结果、图表和笔记全部都放在一个文件里（在数据分析、机器学习、教学和科研等领域的数据实验室）



2.pandas基础

- Pandas是一个功能强大的结构化数据分析的工具集，底层是基于Numpy构建的，无论是在数据分析领域、还是大数据开发场景中都有显著的优势。
- 官网: <https://pandas.pydata.org>
- 核心: DataFrame (类似表格)、Series (类似表格中的一列)

	Name	Age	Sex
0	Jay	18	male
1	Leo	22	male
2	Lily	25	female
3	Jerry	19	male
4	Cici	21	female

index column

DataFrame

Series

小结:

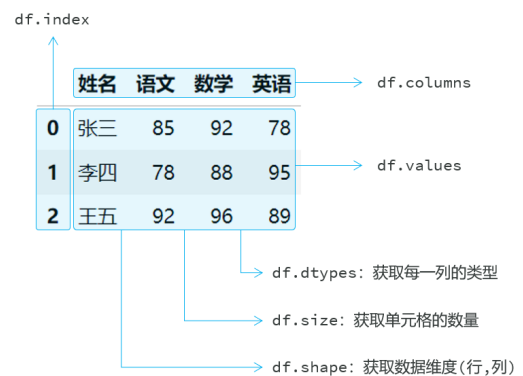
什么是DataFrame, 什么是Series?

- DataFrame是一个表格型的数据结构，就像一张Excel表格，有行有列；
- Series是一列数据，就像DataFrame中的单独一列；

2.1 DataFrame

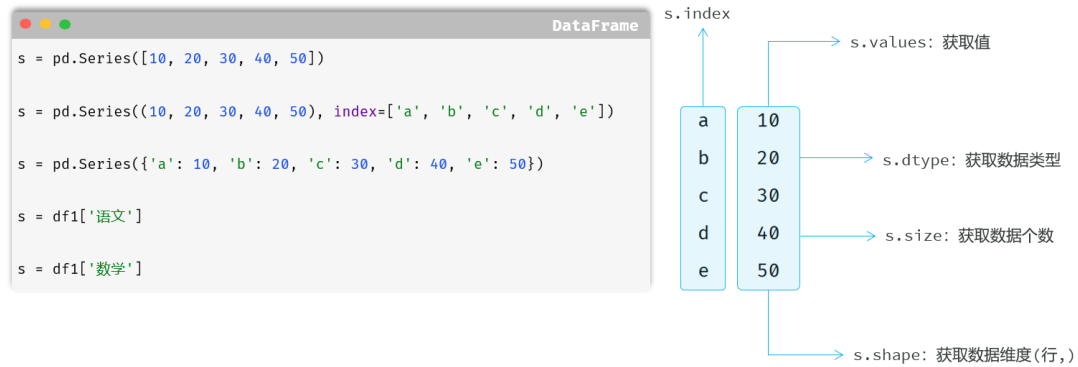
- 构建DataFrame的方式有很多，主要方式如下：

```
df1 = pd.DataFrame([
    {'姓名': '张三', '语文': 85, '数学': 92, '英语': 78},
    {'姓名': '李四', '语文': 78, '数学': 88, '英语': 95},
    {'姓名': '王五', '语文': 92, '数学': 96, '英语': 89}
])
df2 = pd.DataFrame({
    '姓名': ['张三', '李四', '王五'],
    '语文': [85, 78, 92],
    '数学': [92, 88, 96],
    '英语': [78, 95, 89]
})
df3 = pd.DataFrame([
    ('张三', 85, 92, 78),
    ('李四', 78, 88, 95),
    ('王五', 92, 96, 89)
], columns=['姓名', '语文', '数学', '英语'])
df4 = pd.DataFrame([
    ['张三', 85, 92, 78],
    ['李四', 78, 88, 95],
    ['王五', 92, 96, 89]
], columns=['姓名', '语文', '数学', '英语'], index=['a', 'b', 'c'])
```



2.2 Series

- 构建Series的方式有很多，主要方式如下：



小结：

DataFrame、Series中的常见属性的作用？

- xx.index：获取索引
- xx.values：获取值
- xx.dtype：获取数据类型
- xx.size：获取数据个数
- xx.shape：获取数据维度（行，列）
- xx.columns：获取列名（DataFrame特有属性）

2.3 数据读取和写入

- 基于Pandas中提供的API，可以很方便的各类数据文件（csv、Excel、数据库、网络数据等）的读取和写入。



```
import pandas as pd

df = pd.read_csv('../data/sales.csv', usecols=['订单日期', '产品类别', '销售数量', '单价', '客户所在城市', '支付方式'])
df['销售额'] = df['销售数量'] * df['单价']
df.head(20).to_csv('../data/sales_02.csv', index=False)
```

小结：

读取数据和写入数据分别调用什么方法？

- read_xxx，比如：read_csv, read_excel
- to_xxx，比如：to_csv, to_excel

2.4 数据查看、选择及过滤

`df.head(n)`: 查看前n行数据

`df.tail(n)`: 查看结尾n行数据

`df.describe()`: 数值列的统计描述

`df.info()`: 查看数据信息 (列名、非空计数、数据类型)

`df.shape`: 属性, 查看数据维度 (行数, 列数)

`df.columns`: 属性, 查看列名

`df['列名']` / `df.列名`: 操作单列

`df[['列1', '列2']]`: 操作多列

`df.iloc[start:stop:step]`: 基于行号完成切片(不含stop)

`df.loc[start:stop:step]`: 基于索引标签完成切片(含stop)

单条件: `df[df['销售数量'] >= 10]`

单条件: `df[df['产品类别'].isin(['服装', '食品'])]`

单条件: `df[df['单价'].between(50, 200)]`

多条件: `&` (并且)、`|` (或)

这个between默认包含双边界, 可以通过设置inclusive修改

1.数据的查看与选择?

•查看: `head()`、`tail()`、`describe()`、`info()`

•选择列: `df['列名']` / `df[['列名1', '列名2']]`

•选择行: `df.iloc[start:stop:step]` / `df.loc[start:stop:step]`

2.数据的过滤操作?

•语法: `df[条件表达式]`

•示例: `df[df['单价'] >= 100] / df[df['类别'].isin(['.', ''])]`

展示出全部的信息:

配置项，展示所有的数据

```
pd.set_option('display.max_rows', None)
```

2.5 数据清洗

主要对缺失值，重复值，异常值，重复数据进行清理

```
import pandas as pd

pd.set_option('display.min_rows', 30) # 设置显示行数(默认10行)

# 1. 读取数据
df = pd.read_csv('data/sales.csv')

# 2. 数据清洗
# 2.1 缺失值处理
# df.isnull() # 查看缺失值

# 2.1.1 删除缺失值
# df.dropna() # 删除缺失值所在行
# df = df.dropna(axis=1) # 删除缺失值所在列
# df

# 2.1.2 填充缺失值
# df.fillna('--') # 填充缺失值
# df.ffill() # 填充缺失值，填充上一行数据
# df.bfill() # 填充缺失值，填充下一行数据

# 2.2 重复值处理
# 2.2.1 查看重复值
df.duplicated() # 查看重复值(所有的列的数据都重复)
df.duplicated(subset=['订单号']) # 查看重复值(指定列的数据重复)

# 2.2.2 删除重复值
df.drop_duplicates(subset=['订单号']) # keep='last' 保留重复值最后一行；keep='first'
保留重复值第一行；

# 2.3 异常值处理
# 2.3.1 查看异常值
df[df['单价'] < 0]

# 2.3.2 删除异常值
df.drop(df[df['单价'] < 0].index)

# 2.3.3 修复异常值
df['单价'] = df['单价'].abs() # 绝对值
df

# 2.4 数据格式处理
df['订单日期'] = df['订单日期'].str.replace('/', '-')
df
```

2.6 数据排序与分组

2.6.1 数据排序

- 在进行数据排序时，有两种排序方式，分别是：升序和降序。而基于Pandas进行数据排序时，是可以按照多个列进行排序的。

降序: `df.sort_values('销售数量', ascending=False)`

订单号	产品类别	产品名称	销售数量	单价
0	6805677496	家居用品	枕头	10 199
1	5857234132	食品	海苔	5 29
2	5888085059	电子产品	智能音箱	5 399
3	6823473137	食品	面包	3 29
4	6801664364	食品	薯片	2 19
5	5897064217	图书	字帖	4 29
6	6857752388	服装	袜子	7 29
7	6898807429	服装	T恤	6 199
8	5875402232	电子产品	路由器	9 299
9	5888085066	图书	字典	2 99

订单号	产品类别	产品名称	销售数量	单价
0	6805677496	家居用品	枕头	10 199
8	5875402232	电子产品	路由器	9 299
6	6857752388	服装	袜子	7 29
7	6898807429	服装	T恤	6 199
1	5857234132	食品	海苔	5 29
2	5888085059	电子产品	智能音箱	5 399
5	5897064217	图书	字帖	4 29
3	6823473137	食品	面包	3 29
4	6801664364	食品	薯片	2 19
9	5888085066	图书	字典	2 99

订单号	产品类别	产品名称	销售数量	单价
4	6801664364	食品	薯片	2 19
9	5888085066	图书	字典	2 99
3	6823473137	食品	面包	3 29
5	5897064217	图书	字帖	4 29
2	5888085059	电子产品	智能音箱	5 399
1	5857234132	食品	海苔	5 29
7	6898807429	服装	T恤	6 199
6	6857752388	服装	袜子	7 29
8	5875402232	电子产品	路由器	9 299
0	6805677496	家居用品	枕头	10 199

升序: `df.sort_values('销售数量', ascending=True)`

`df.sort_values(['销售数量', '单价'], ascending=[True, True])`

多列排序: 进行多个列排序时，会先按照第一列进行排序，第一列的值相同时，才会按照第二列进行排序。

2.6.2 数据分组

- 分组操作就是把数据按照某个特征分成不同的组，然后对每个组分别进行统计计算。

订单号	产品类别	产品名称	销售数量	单价	销售额
0	6805677496	家居用品	枕头	10	1990
1	5857234132	食品	海苔	5	145
2	5888085059	电子产品	智能音箱	5	1995
3	6823473137	食品	面包	3	87
4	6801664364	食品	薯片	2	38
5	5897064217	图书	字帖	4	116
6	6857752388	服装	袜子	7	203
7	6898807429	服装	T恤	6	1194
8	5875402232	电子产品	路由器	9	2691
9	5888085066	图书	字典	2	198

订单号	产品类别	产品名称	销售数量	单价	销售额
0	6805677496	家居用品	枕头	10	1990
1	5857234132	食品	海苔	5	145
3	6823473137	食品	面包	3	87
4	6801664364	食品	薯片	2	38
2	5888085059	电子产品	智能音箱	5	1995
8	5875402232	电子产品	路由器	9	2691
5	5897064217	图书	字帖	4	116
9	5888085066	图书	字典	2	198
6	6857752388	服装	袜子	7	203
7	6898807429	服装	T恤	6	1194

分组后求和 `df.groupby('产品类别')['销售额'].sum()`

分组后统计数量 `df.groupby('产品类别')['销售额'].count()`

分组后求最大值 `df.groupby('产品类别')['销售额'].max()`

分组后求最小值 `df.groupby('产品类别')['销售额'].min()`

分组后求平均值 `df.groupby('产品类别')['销售额'].mean()`

`df.groupby('产品类别')['销售额'].agg(['sum', 'count', 'max', 'min', 'mean'])`

```
df = pd.read_csv('data/sales.csv', nrows=20)
```

```
df['销售金额'] = df['单价'] * df['销售数量']
```

3. 分组

3.1 根据 产品类别 分组，统计各个类别的 订单数量 - count

```
df.groupby('产品类别')['订单号'].count()
```

3.2 根据 产品类别 分组，统计各个类别的 销售数量 之和 - sum

```
df.groupby('产品类别')['销售数量'].sum()
```

3.3 根据 产品类别 分组，统计各个类别的 销售金额 之和 - sum

```
df.groupby('产品类别')['销售金额'].sum()
```

3.4 根据 产品类别 分组，统计各个类别的最低商品 单价 - min

```
df.groupby('产品类别')['单价'].min()
```

3.5 根据 产品类别 分组，统计各个类别的最高商品 单价 - max

```
df.groupby('产品类别')['单价'].max()
```

3.6 根据 产品类别 分组，统计各个类别的平均商品 单价 - mean

```
df.groupby('产品类别')['单价'].mean()
```

3.7 根据 产品类别 分组，统计各个类别的商品的 平均单价、最高单价、最低单价 - agg

```
df.groupby('产品类别')['单价'].agg(['mean', 'max', 'min'])
```

```
df.groupby('产品类别').agg({'单价': ['mean', 'max', 'min']})
```

3.8 根据 产品类别 分组，统计各个类别的商品的 销售数量 之和，销售金额 之和，平均 单价

```
df.groupby('产品类别').agg({'销售数量': 'sum', '销售金额': 'sum', '单价': 'mean'})
```

3. Matplotlib

- Matplotlib是一个功能强大的数据可视化开源Python库，也是Python中使用的最多的图形绘图库，可以创建静态、动态、交互式的图表。

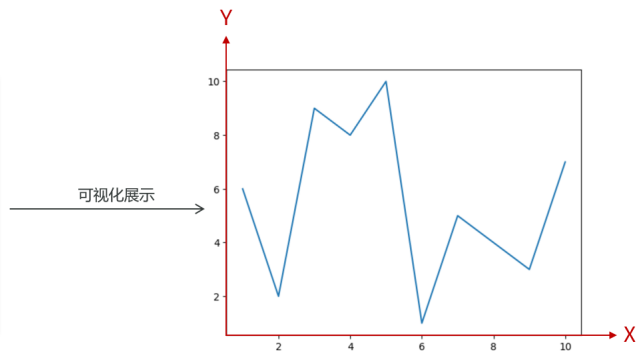
- 官网: <https://matplotlib.org>

- 安装: `pip install matplotlib`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [6, 2, 9, 8, 10, 1, 5, 4, 3, 7]

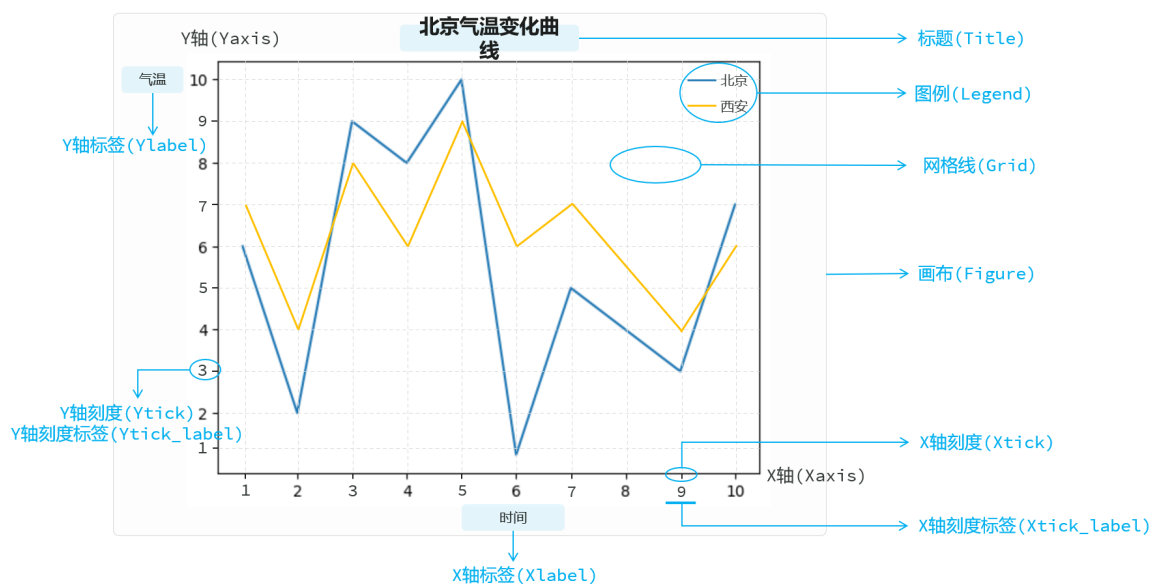
plt.plot(x, y)
plt.show()
```



注意：X轴的数据数量与Y轴中的数据数量要一致。

3.1 图表详解

Matplotlib图表详解



```
import matplotlib.pyplot as plt
import random
```

展示中文

```
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体

x = [i for i in range(1, 25)]
y_bj = [random.randint(10,15) for i in x]
y_xa = [random.randint(13,18) for i in x]

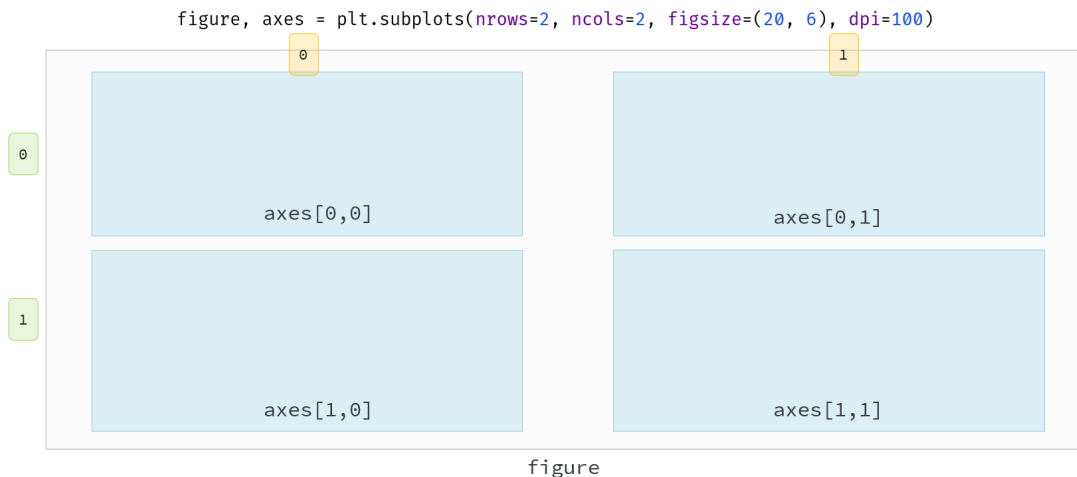
plt.figure(figsize=(10, 5)) # 设置画布大小，宽，高
plt.plot(x, y_bj, label='北京') # 折线图 -> 如果没有画布，会自动创建一个画布
plt.plot(x, y_xa, label='西安')

# 设置折线图的详细信息
plt.title('气温变化折线图', fontsize=15) # 标题
plt.xlabel('时间') # x轴标签
plt.ylabel('温度') # y轴标签
# plt.xticks(x[::2])
# plt.xticks(x[1::2])
plt.xticks(x) # x轴刻度
y_ticks = [i for i in range(5,21)]
plt.yticks(y_ticks) # y轴刻度
plt.grid(linestyle='--', alpha=0.3) # 显示网格
plt.legend(loc='upper right') # 显示图例

plt.show() # 显示图表
```

3.2 创建子图

- 为了能同时展示多个图表，便于图表之间数据的直观对比和分析，更高效、更专业地组织和呈现复杂的可视化信息，通常会在一个画布上创建多个子图。



```
from matplotlib.axes import Axes
import matplotlib.pyplot as plt

# 展示中文
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体

# 创建子图
# figure: 画布对象 ; axes: 子图数组(里面存放的是 Axes 类型的对象)
figure, axes = plt.subplots(nrows=1, ncols=2, figsize=(20,6), dpi=100) # nrows: 行, ncols: 列

# 图一：柱状图（世界石油储备） - bar方法
```

```

countries = ['中国', '美国', '印度', '加拿大', '伊拉克', '沙特', '伊朗', '英国', '德国'] # 国家列表
values = [35, 23, 18, 21, 56, 78, 51, 12, 18]

axes1: Axes = axes[0]
axes1.bar(countries, values, width=0.6, color='g') # 柱状图 - width: 柱子宽度, color: 颜色
axes1.set_title('世界石油储备', fontsize=18) # 设置标题
axes1.set_xlabel('国家') # X轴标签
axes1.set_ylabel('石油储备(亿吨)') # Y轴标签
axes1.grid(linestyle='--', alpha=0.3) # 网格线

# 图二: 饼状图 (世界人口) --- > 擅长比例分析 -- pie 方法
countries2 = ['印度', '中国', '美国', '印尼', '巴基斯坦', '尼日利亚', '巴西', '俄罗斯', '其他']
values = [14.51, 14.09, 3.4, 2.83, 2.51, 2.33, 2.12, 1.44, 20]

axes2: Axes = axes[1]
axes2.pie(values, labels=countries2, autopct='%1.1f%%') # 饼状图
axes2.set_title('世界人口比例', fontsize=18) # 设置标题
axes2.legend(loc='lower center', ncol=5, bbox_to_anchor=(0.5, -0.08)) # 显示图例, ncol: 每行显示的个数, bbox_to_anchor: 图例位置(0.5 控制水平方位, -0.08 控制垂直方位)

# 保存图片
plt.savefig('data/01.png')

plt.show() # 展示图表

```

小结:

1.如何在画布中创建子图?

•figure, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=100)

2.如何绘制折线图、柱状图、饼状图?

•折线图: plot(...) ---> 适合展示趋势变化

•柱状图: bar(...) ---> 适合展示数量对比

•饼状图: pie(...) ---> 适合展示比例构成

案例

```

from matplotlib.axes import Axes
import pandas as pd
import matplotlib.pyplot as plt
from pandas import Series

# 展示中文
plt.rcParams['font.sans-serif'] = ['SimHei']

# 创建子图
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(20, 12), dpi=100)

```

```

fig.suptitle('TMDb-TOP300电影榜单数据统计', fontsize=23, x=0.5, y=0.95) # 添加画布标题
# - x=0.5 : X轴位置; y=0.95 : Y轴位置
fig.subplots_adjust(hspace=0.4, wspace=0.2) # 调整子图间距, hspace: 控制垂直间距,
wspace: 控制水平间距

# 获取子图
axes1: Axes = axes[0][0]
axes2: Axes = axes[0][1]
axes3: Axes = axes[1][0]
axes4: Axes = axes[1][1]

# 加载数据
# int64 : 整型数字 (不支持空值); Int64 : 整型数字 (支持空值); float64 : 浮点型数字 (支持空值);
data = pd.read_csv('data/movies.csv', usecols=['电影名', '年份', '上映时间', '类型', '时长', '评分', '语言'], dtype={'年份': 'Int64'})

# 1. 需求一: 统计TOP300的电影中, 每一年上映的电影数量的变化。(折线图)
# 1.1 缺失值、异常值处理
# data.isnull().sum()
data['年份'] = data['年份'].fillna(data['上映时间'].str[:4])

# 1.2 分组统计
year_count = data.groupby('年份')['年份'].count()

# 1.3 组装数据
# x轴数据
min_year = year_count.index.min()
max_year = year_count.index.max()
x = [i for i in range(min_year, max_year+1)]

# y轴数据
y = [int(year_count.get(i, 0)) for i in x]

# 1.4 绘制折线图
axes1.plot(x, y, color='green') # 折线图
axes1.set_title('每年电影数量变化折线图', fontsize=18) # 添加子图标题
axes1.set_xlabel('年份', fontsize=12) # 添加X轴标签
axes1.set_ylabel('电影数量', fontsize=12) # 添加Y轴标签

axes1.set_xticks(x[:8]) # 设置X轴刻度间隔
y_ticks = [i for i in range(0, 31, 3)]
axes1.set_yticks(y_ticks) # 设置Y轴刻度间隔
axes1.grid(linestyle='--', alpha=0.5) # 添加网格线

# 2. 需求二: 统计对比不同语言的电影数量。(柱状图)
# 2.1 获取不同语言对应的电影数量
language_count = data.groupby('语言')['语言'].count().sort_values(ascending=False)

x_language = language_count.index.tolist()

```

```
y_language_count = language_count.values.tolist()
```

2.2 绘制柱状图

```
axes2.bar(x_language, y_language_count, color='green', width=0.7) # 柱状图
axes2.set_title('不同语言电影数量柱状图', fontsize=18) # 添加子图标题
axes2.set_xlabel('语言', fontsize=12) # 添加X轴标签
axes2.set_ylabel('电影数量', fontsize=12) # 添加Y轴标签
axes2.grid(linestyle='--', alpha=0.5) # 添加网格线
axes2.tick_params(axis='x', rotation=90) # 旋转X轴标签
```

3. 需求三：统计对比不同类型电影数量（柱状图）

3.1 获取不同类型对应的电影数量

```
type_count = {} # {'剧情': 5, '犯罪': 3}
for types in data['类型'].str.split(','): # [剧情, 犯罪]
    for t in types: # 剧情
        if t in type_count:
            type_count[t] += 1
        else:
            type_count[t] = 1
```

```
x_types = list(type_count.keys()) # 类型列表
```

```
y_values = list(type_count.values()) # 数量列表
```

3.2 绘制柱状图

```
axes3.bar(x_types, y_values, color='green', width=0.7) # 柱状图
axes3.set_title('不同类型电影数量柱状图', fontsize=18) # 添加子图标题
axes3.set_xlabel('类型', fontsize=12) # 添加X轴标签
axes3.set_ylabel('电影数量', fontsize=12) # 添加Y轴标签
axes3.grid(linestyle='--', alpha=0.5) # 添加网格线
axes3.tick_params(axis='x', rotation=90) # 旋转X轴标签
```

4. 需求四：统计不同评分电影数量占比（饼状图）

4.1 获取不同评分对应的电影数量

```
score_count = data.groupby('评分')['评分'].count()
```

```
# 合并小数据(比例 < 2%) --> 其他
```

```
total = score_count.sum()
```

```
large_scores: Series = score_count.loc[score_count >= total*0.02] # 大数据，比例 >= 2%
```

```
small_scores: Series = score_count.loc[score_count < total*0.02] # 小数据，比例 < 2%
```

```
if small_scores.shape[0] > 0:
    large_scores['其他'] = small_scores.sum()
```

```
scores = large_scores.index.tolist() # 评分列表
```

```
scores_values = large_scores.values.tolist() # 数量列表
```

4.2 绘制饼状图

```
axes4.pie(scores_values, labels=scores, autopct='%1.1f%%', startangle=0,
radius=1.2) # startangle 起始角度
```

```
axes4.set_title('不同评分电影数量占比饼状图', fontsize=18)
axes4.legend(loc='lower center', ncol=4, bbox_to_anchor=(0.5, -0.2))

# 5. 保存图片
plt.savefig('data/TMDB-TOP300.png')

plt.show() # 显示画布
```

五、Web开发

1.FastAPI

- FastAPI是一个现代、快速、高性能的Web框架，用于基于标准的Python类型提示构建API接口服务。
- 官网：<https://fastapi.org.cn>



API接口：应用程序编程接口(Application Programming Interface)，就是对外提供的功能入口，供别人来调用（比如：[天气查询的API接口](#)）。

FastAPI入门

- 使用步骤：

1. 导入FastAPI
2. 创建FastAPI实例对象
3. 创建路径操作函数，定义访问路径
4. 运行FastAPI服务

`fastapi dev "xxx.py"`
`uvicorn xxx:app --reload`

```
main.py
from fastapi import FastAPI

# 创建FastAPI实例
app = FastAPI()

# 定义功能接口
@app.get("/")
def root():
    return {"message": "Hello World"}

# 定义功能接口
@app.get("/users")
def get_users():
    return [{"id": 1, "name": "Alice", "age": 18},
            {"id": 2, "name": "Bob", "age": 20}]

# 启动FastAPI服务器
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

uvicorn: 专门为现代 Python Web 框架 (FastAPI、Starlette) 设计的高性能服务器。

小结：

1.什么是的FastAPI?

•FastAPI是一个现代、快速、高性能的Python Web框架，用来构建API接口服务

2.基于FastAPI如何来开发服务端接口？

•导入FastAPI

•创建FastAPI实例对象

•创建路径操作函数，定义访问路径

•运行FastAPI服务

2.RESTful

Restful

- Restful指的是遵循REST架构风格的API接口服务，而REST (REpresentational State T Transfer)，表述性状态转换，它是一种软件架构风格。

REST风格url	请求方式	含义	备注
http://localhost:8000/users/1	GET	查询id为1的用户	URL定位资源 HTTP动词描述操作 简洁、规范、优雅
http://localhost:8000/users/1	DELETE	删除id为1的用户	
http://localhost:8000/users	POST	新增用户	
http://localhost:8000/users	PUT	修改用户	

注意：REST是风格，是约定方式，约定不是规定，可以打破。

注意：描述功能模块通常使用复数形式（加s），表示此类资源，而非单个资源。如：users、books、items 小结：

1.REST风格的特点？

•URL定义资源

•HTTP动词描述操作

2.REST风格中的四种请求方式及对应的操作？

•GET：查询

•POST：新增

•PUT：修改

•DELETE：删除

3.文件挂载

基础环境搭建

1. 创建项目文件夹，将资料中的 static 目录拷贝到项目中。
2. 编写python程序，定义路径操作函数，访问前端HTML页面。

main.py

```
from fastapi import FastAPI
from fastapi.responses import FileResponse

# 创建FastAPI实例
app = FastAPI(title="汉字谜盒")

# 挂载静态文件目录
app.mount("/static", StaticFiles(directory="static"), name="static")

# 定义路径操作函数
@app.get("/")
def root():
    return FileResponse("static/index.html")
```

任何以/static开头的路径都会交给这儿处理

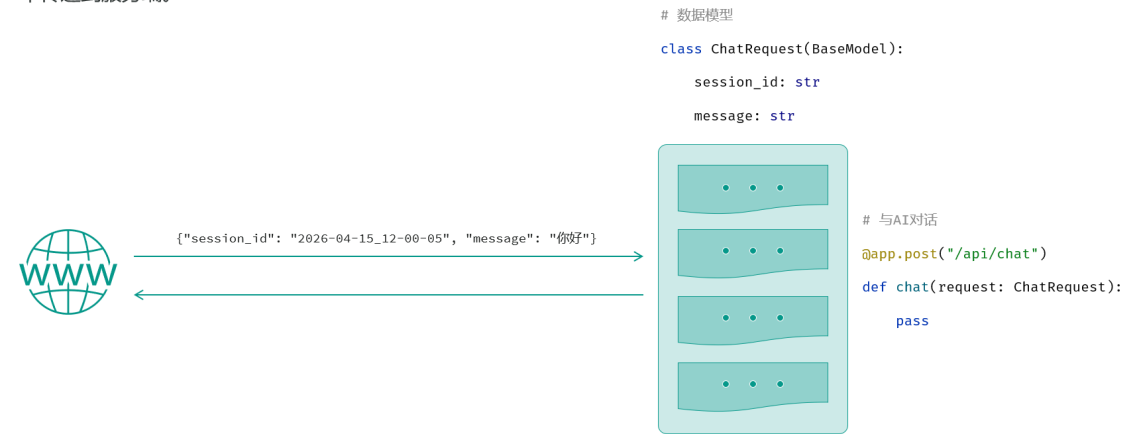
FastAPI内部使用的名称(可以自由定义)

HTML、CSS、JS等静态资源存放目录

4.post请求格式

与AI交互

- 在与AI进行交互式，前端传递给服务端的参数包含两项，分别为session_id 与 message，并以json格式在请求体中传递到服务端。



BaseModel：是Pydantic库提供的父类（FastAPI 深度集成了 Pydantic），用于定义FastAPI数据模型和数据验证规则。

也即是实现了dto层

5.日志记录

- 为了能够灵活的控制项目中日志的输出，我们可以通过官方提供的logging模块来输出日志，具体做法如下：

```
from fastapi import FastAPI
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [(filename)s:%(lineno)d] - %(message)s"
)

# 创建FastAPI实例
app = FastAPI()

# 定义路径操作函数
@app.get("/")
def root():
    logging.info("访问首页~")
    return FileResponse("static/index.html")
```

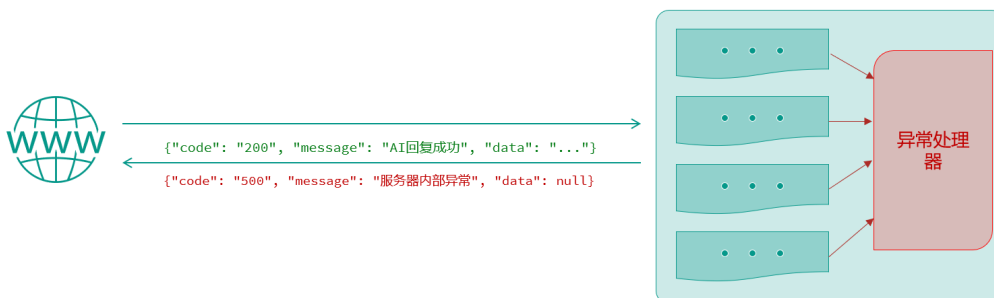
```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [(filename)s:%(lineno)d] - %(message)s"
)
```

日志级别：日志级别就是给日志信息贴上的"重要性标签"，常见的级别有：DEBUG、INFO、WARNING、ERROR、FATAL（日志级别依次升高）。

6.统一异常处理

- 项目中的功能较多，目前我们并未考虑异常处理，可以借助于FastAPI中的统一异常处理方案来处理异常。

```
# 统一处理异常信息
@app.exception_handler(Exception)
def handle_exception(request: Request, exc: Exception):
    logging.error(f"处理异常，请求路径: {request.url}, 异常信息: {exc}")
    return JSONResponse(content={"code": 500, "message": "服务器内部错误"})
```



```
# 统一处理异常信息
@app.exception_handler(Exception)
def handle_exception(request: Request, exc: Exception):
    logging.error(f"处理异常，请求路径: {request.url}, 异常信息: {exc}")
    return JSONResponse(content={"code": 500, "message": "服务器内部错误"})
```

